README Open-Source Python Software Deep Learning Automated Detection of Severe Storm Signatures Within Geostationary Satellite Imagery

John W. Cooney

September 8, 2026

Contents

1 *Document Overview* **5**

2 *Installing Software and Creating Python Environment* **6**

2.1 *Mac/Linux/Unix Users* 6

2.2 *Windows Users* 8

2.3 *Version 5.0.0 Existing Users Python Environment* 9

2.4 *Software Path Setup* 9

3 *Running the Model Software* **10**

3.1 *How to Run the Software* 10

3.2 *Model Input Data Acronyms* 11

3.3 *Configuration File* 11

3.4 *Optimal Models* 14

3.4.1 *OT Optimal Model From Testing* 14

3.4.2 *AACP Optimal Model From Testing* 15

3.4.3 *Version 2 Software and Older* 15

3.5 *Model Checkpoint Files* 15

3.6 *Script Breakdown* . . . 16
3.7 *Model Input/Output Files and Locations* . . . 19

4 *Model Characteristics* **20**
4.1 *Model Training* . . . 20
4.2 *Model Types* . . . 21
4.2.1 *OTs* . . . 21
4.2.2 *AACPs* . . . 22
4.3 *Allowed Data Ranges* . . . 22
4.4 *Optimal Model Likelihood Scores & Thresholds* . . . 22

5 *Model Input Data* **23**
5.1 *Model Satellite Data Inputs* . . . 23
5.2 *Data Download Sources* . . . 26
5.2.1 *Google Cloud* . . . 26
5.2.2 *Additional GOES Resources* . . . 27
5.2.3 *EUMETSAT Data Services* . . . 27

6 *Output netCDF Files* **28**
6.1 *Overview* . . . 28
6.2 *Script Breakdown* . . . 28
6.3 *Path Setup* . . . 29
6.4 *Removed Data* . . . 30

7 *Creating Numpy Arrays for Model Input* **30**
7.1 *Overview* . . . 30
7.2 *Script Breakdown* . . . 31

8 *Post-Processing Model Results* **33**
8.1 *Overview* . . . 33
8.2 *Script Breakdown* . . . 34
8.3 *Post-Processing Variables and Calculations* . . . 34

8.3.1 *Object Identification Numbers* . . . 34
8.3.2 *IR OT-Anvil Brightness Temperature Difference* . . . 35
8.3.3 *Tropopause Temperature* . . . 35

9 *Google Cloud Platform (GCP)* **36**
9.1 *Overview* . . . 36
9.2 *Installing Google Cloud* . . . 37
9.3 *Google Cloud Problems Users May Encounter* . . . 39
9.3.1 *Local Web Proxy/Firewall Error* . . . 39
9.4 *Script Breakdown* . . . 39

10 *Software Updates* **40**
10.1 *MAJOR REVISIONS* . . . 40
10.1.1 *05 June 2023* . . . 40
10.1.2 *22 June 2023* . . . 40
10.1.3 *23 August 2023* . . . 40
10.1.4 *6 March 2024 VERSION 2 Release* . . . 40
10.1.5 *2 May 2024* . . . 41
10.1.6 *9 January 2025* . . . 42
10.1.7 *16 April 2025 VERSION 3 Release* . . . 42
10.1.8 *30 July 2025 VERSION 3.3.0 Release* . . . 43
10.1.9 *17 April 2026 VERSION 4.0.0 Release* . . . 44
10.1.10 *8 September 2026 VERSION 5.0.0 Release* . . . 45
10.2 *MINOR REVISIONS* . . . 46
10.2.1 *13 June 2023* . . . 46
10.2.2 *21 August 2023* . . . 46
10.2.3 *2 May 2024* . . . 47
10.2.4 *20 May 2025* . . . 47
10.2.5 *28 April 2026* . . . 47
10.2.6 *07 May 2026* . . . 47
10.2.7 *12 June 2026* . . . 48

11 *Authors* **48**

11.1 *Contact Information* 48

12 *Disclaimer and Copyright Notices* **48**

12.1 *Notices* 48

12.2 *Disclaimers* 49

12.3 *Waiver and Indemnity* 49

13 *Appendix* **50**

13.1 *Allowed Satellites* 50

13.2 *Anaconda Installation* 50

13.3 *MTG and SEVIRI Satellite Setup* 51

13.4 *netCDF File Contents* 52

13.4.1 *GLM Gridder netCDF File* 52

13.4.2 *Combine IR, VIS, and GLM Data netCDF File* 55

13.5 *GOES Storage Bucket Download Examples* 57

13.6 *Potential Speed Ups* 57

13.7 *ProcessPool Adjustments for Creating Combined netCDF Files* 57

13.8 *ProcessPoolExecutor Adjustments for Post-Processing* 58

14 References Cited **59**

1 *Document Overview*

This document is intended to help the user install and run the open source software to detect severe storm signatures in geostationary imagery. The first thing the user will want to do is install Anaconda (Section 13.2). Anaconda allows the user to install many of the necessary Python modules in a conda environment so that all of the python modules are always compatible. This document will walk the user through the entire software setup as well as show the user how the model can be run. The software was setup using Mac/linux OS but should work using Windows OS as well. All code within this document is capable of being run entirely on the Google Cloud Platform (Section 9) using Google's Virtual Machines (VMs), but Google is not necessary to run the model.

VERSION 5.0.0 of software was released on 8 September 2026!! Added SEVIRI data to software OT detection capabilities! Software is now able to handle mtg1, seviri, seviri-iodc, msg1, msg2, msg3, and msg4 satellites. This is a major overhaul of the software framework due to ability to now use python module satpy (license recently changed allowing us to use). See Section 10 to see all updates. Existing users wishing to run the software for SEVIRI will need to create a new conda environment that they run the code out of. The reason for this is due to satpy not being compatible with the older modules that were installed previously.

VERSION 5.0.0 of software was released on 8 September 2026!! SEVIRI data is now fully integrated into the software's OT detection capabilities! The software can now automatically handle `mtg1`, `seviri`, `seviri-iodc`, `msg1`, `msg2`, `msg3`, and `msg4` satellite inputs. This represents a major architectural overhaul to the software framework due to the integration of the Python `satpy` module (whose license was recently changed, allowing us to utilize it natively). See Section 10 to see all updates. **IMPORTANT: Existing users MUST create a new conda environment (`svrstormsig_v5`) to run this update.** This is because `satpy` and our newly upgraded machine learning backend (Python 3.10 and TensorFlow 2.15) are fundamentally incompatible with the older modules installed in previous software versions. Existing users, please 2.3 for how to install.

VERSION 4.0.0 of software was released on 17 April 2026!! Many user requested updates have been completed. The user can now run OT and AACP model simultaneously in both real-time and archived modes! When run in this mode, the default optimal thresholds and models will

be used to detect OTs and AACPs. See Section 10 to see all updates but AACP post-processing saw a significant overhaul! We STRONGLY recommend using AACP object IDs when making identifications instead of the raw AACP probabilities output by the model. This is due to all of the post-processing clean up implementations in this new release. We significantly cut false alarms, particularly some of the edge artifacts we were seeing. In this ReadMe document, we also are now identifying places that users could change to speed up run times (see Section 13.6). We did not apply the run time enhancements in the software release because they are dependent on a users computer specs. The speed ups make use of multi-processing in parallel threads so people should consider the performance of the server or computer that the software is being run on before applying them.

Software Users prior to 8 September 2026 will need to pull the latest python files from GitHub (see 2.3). This should be very fast. Users MUST also download the new checkpoint and MTG intermediate files from 'https://science-data.larc.nasa.gov/LaRC-SD-Publications/2023-05-05-001-JWC/data/ML_data.zip'. Once downloaded and unzipped, replace the old model_checkpoints subdirectory with the latest one that was just downloaded. This directory includes all of the improved model detection files as well as checkpoint files for the new TROPDIFF input OT model for SEVIRI. If you have issues downloading and unzipping the file from the browser, you can run the run_download_model_chkpoint_files.py script. Users have experienced issues downloading the checkpoint files directly from the browser and this script has been successful.

See Cooney et al. (2025) for information and citations.

2 *Installing Software and Creating Python Environment*

IMPORTANT NOTE!!! Software could take an hour to install. The package is setup to grab netCDF look up table files that are stored on a NASA server for faster run jobs.

2.1 *Mac/Linux/Unix Users*

There are 11 steps for installing the software. First, the user MUST have Anaconda installed (Section 13.2). Do not do the following without first installing Anaconda. This software package requires the use of the glmtools and lmatools libraries. The steps below will install these GitHub

repositories for you. Please follow the steps in order for a successful installation.

1. the user must clone the project repository *git clone https://github.com/nasa/svrstormsig.git* in Terminal.
2. cd svrstormsig
3. git clone https://github.com/deeplycloudy/glmtools.git
4. cd glmtools
5. cp ../environment.yml .
6. cp ../setup.py .
7. conda env create -f environment.yml
8. conda activate svrstormsig_v5
9. pip install -e .
10. Refer to Section 9.2 for setting up Google Cloud to download data files from there. This service is free and only requires users have a Gmail account. If you do not wish to download data from Google Cloud, see 5.2.2 for more information regarding where to obtain the necessary data.
11. Refer to Section 13.3 for instructions on how to set up the ability to download MTG or SEVIRI data files from EUMETSAT data services. This service is free and easy to use. If you desire to run software in real-time mode though, your account must have a EUMETSAT subscription.

Once the steps are complete you should be able to successfully run the software. The package can then be used when run within the svrstormsig environment. In order to activate the svrstormsig environment, type *conda activate svrstormsig_v5* into the Terminal. This environment lets you use all of the installed libraries for this software. The package can also be used when run as a kernel within the svrstormsig environment. This kernel is created by running: python -m ipykernel install –user –name=svrstormsig. Note the difference between 1 hyphen in front of m and 2 hyphens in front of user and name. If ipykernel is not installed then run: conda install ipykernel and try again.

2.2 *Windows Users*

Note, one method Windows users could do is install a Linux Virtual environment and run software there. If that is not what you want to do, the software should work on Windows devices.

IMPORTANT NOTE: Windows users must have 64-bit installed!!! Tensorflow, a main component of machine learning, will not work otherwise.

I apologize in advance if this section is not written as well as others. I am not a Windows user and so I am not familiar with all of the nuances available to Windows operators. These are the 11 steps I found to work for installing the software on Windows. You may have a method that works more efficiently. If so, please email me so that I can better assist Windows users. First, the user MUST have Anaconda installed (Section 13.2). Do not do the following without first installing Anaconda. This software package requires the use of the glmtools and lmatools libraries. The steps below will install these GitHub repositories for you. Please follow the steps in order for a successful installation.

1. the user must clone the project repository *git clone https://github.com/nasa/svrstormsig.git* in the Anaconda Powershell Prompt.
2. cd svrstormsig
3. git clone https://github.com/deeplycloudy/glmtools.git
4. cd glmtools
5. copy ..\windows_environment.yml .\environment.yml
6. copy ..\setup.py .
7. conda env create -f environment.yml
8. conda activate svrstormsig_v5
9. pip install -e .
10. Refer to Section 9.2 for setting up Google Cloud to download data files from there. Follow steps 1-15 outlined. For step 16, I am unsure about .bash_profiles or .zshrc on Windows at this time, so instead in the Anaconda Powershell Prompt I found typing this works:

$env:GOOGLE_APPLICATION_CREDENTIALS='key_path'

where key_path is the local path and filename to the json key file. This service is free and only requires users have a gmail account. If you do not wish to download data from Google Cloud, see 5.2.2 for more information about where to obtain the necessary data.

11. Refer to Section 13.3 for instructions on how to set up the ability to download MTG or SEVIRI data files from EUMETSAT data services. This service is free and easy to use. If you desire to run software in real-time mode though, your account must have a EUMETSAT subscription.

2.3 *Version 5.0.0 Existing Users Python Environment*

For existing users, one must simply do a git pull followed by conda env create -f environment.yml. Users will then access that new environment when they want to run the software. The steps are outlined below.

1. cd svrstormsig
2. git pull origin main
3. conda env create -f environment.yml
4. conda activate svrstormsig_v5
5. python update_weights.py

Once the steps are complete you should be able to successfully run the software in the svrstormsig_v5 environment.

2.4 *Software Path Setup*

This section displays what the directory and subdirectory structure of the software should look like after install. If the directory path structure is not setup like this, then you may run into errors when attempting to run the software.

Directory Structure:

$\longrightarrow$ svrstormsig

⟶ data

⟶ model_checkpoints

⟶ region

⟶ sat_projection_files

⟶ glmtools

⟶ glmtools

⟶ glmtools_docs

⟶ glmtools.egg-info

⟶ python

⟶ EDA

⟶ glm_gridder

⟶ gridrad

⟶ model

⟶ new_model

⟶ visualize_results

3 *Running the Model Software*

This section will discuss how to run the software, optimal models, script breakdown, and the directory path setup.

3.1 *How to Run the Software*

First, activate your conda environment in the Terminal by typing *conda activate svrstormsig*. This will put you into the environment that has all of the required python modules. All of the software is built and run using Python code. The main script to run the code is run_all_plume_updraft_model_predict.py and is located in the new_model subdirectory. The easiest 2 methods to run the software are listed below. The first method allows the user to answer prompts in the Terminal to determine what model they want to run, the model inputs, and the dates they want to run the model over. The second method takes those prompts and puts them into a more detailed and easy to use configuration file. The configuration file can then be loaded into the software to specify the user's model run

needs. See Section 3.3 for more details on the configuration file setup. The default configuration is provided at the time of software install. Note the 2 hyphens used in front of config.

1. python3 path-to-run-script/run_all_plume_updraft_model_predict.py

2. python3 path-to-run-script/run_all_plume_updraft_model_predict.py –config path-to-configuration-file/modrun_config.cfg

For users with raw MTG/MSG data in the format yyyy/mm/dd/ run the software with the essl keyword set (ex. python3 path-to-run-script/run_all_plume_updraft_model_predict.py –config path-to-configuration-file/modrun_config.cfg –essl)

3.2 *Model Input Data Acronyms*

IR refers to the clean infrared window (10.3μm) brightness temperatures.

VIS refers to the visible reflectance window (0.64μm).

GLM refers to the Geostationary Lightning Mapper flash extent density data.

WVIRDIFF (6.2μm - 10.3μm) refers to the difference between the water vapor window (6.2μm) and the clean IR window brightness temperatures.

TROPDIFF (10.3μm - GFS TropT) refers to the difference between the clean IR window brightness temperature and the GFS tropopause temperature.

SNOWICE (1.6μm) refers to the snow/ice channel.

CIRRUS (1.37μm) refers to the cirrus channel.

DIRTYIRDIFF (12.3μm - 10.3μm) refers to the difference between the dirty IR window (12.3μm) brightness temperatures and the clean IR window brightness temperatures.

3.3 *Configuration File*

A default configuration file, modrun_config.cfg, is installed when you clone the project git repository. This file can be changed by the user to suit their needs. Some may find using a configuration file easier than answering prompts in the Terminal window. The contents of the default configuration file are shown below. **Users should ONLY change items to the right of the = sign to suit their model run needs**. If anyone would like additions, clarifications, or changes, please email john.w.cooney@nasa.gov and we will address those changes as soon as possible for the next software release.

```
########################
## Model Run Configuration Settings
##
```

```
#  Number 2 corresponds to real-time run ONLY.
#  Numbers 3-4 correspond to not real-time/archived run ONLY.
#
## IMPORTANT NOTE: ONLY change values on the right side of equals sign!!

#1)
#Is this a real-time run or not? (y/n):
real_time = n


#2)
#Question 2 corresponds to real-time run ONLY. Ignore if real_time in Q1 is n or no
#Type below the number of hours you want the model to run for:
number_of_hours = 12


#3)
#Question 3 corresponds to not-real-time run ONLY. Ignore if real_time in Q1 is y or yes
#Enter the model start date (ex. 2017-04-29 00:00:00):
start_date = 2026-08-10 21:00:00


#4)
#Question 4 corresponds to not-real-time run ONLY. Ignore if real_time in Q1 is y or yes
#Enter the model start date (ex. 2017-04-29 00:00:00):
end_date = 2026-08-11 01:30:00


#5)
#Question 5 corresponds to not-real-time run ONLY. Ignore if real_time in Q1 is y or yes
#Do you need to download the satellite data files? (y/n):
download_data = y


#6)
#If download_data is n or no or if real_time in Q1 is y or yes, ignore this question.
#Question 6 corresponds to not-real-time/archived run ONLY.
#Enter the location of raw data files:
#DEFAULT = None, which implies to use the default file root location in the code.
raw_data_directory = None


#5)
#Type below the name of the satellite you wish to use (ex. goes-16, goes-17, goes-18, goes-19, mtg1, seviri, seviri-iodc, msg1, msg2, msg3, msg4)
satellite = goes-18


#6)
#Type the satellite scan sector you wish to use. If satellite does not have multiple
#scan sectors, ignore this. (ex. M1, M2, F, C)
sector = C


#7)
#Type US state or country you wish to bound model output to.
#DEFAULT = None, so model is not bound to any particular state or country.
#See extract_us_lat_lon_region in rdr_sat_utils_jwc.py for examples. User
#can also add their own regions there.
region = None
```

```
#8)
#Type longitude and latitude boundaries to confine the data to (in degrees):
#[longitude_min, latitude_min, longitude_max, latitude_max]
#(ex. [-100.0, 36.0, -86.0, 42.0])
#DEFAULT = [], so model results output are not restricted to any user
#specified lat/lon bounding box
xy_bounds = []


#9)
#Enter the desired severe weather indicator (OT or AACP or BOTH):
#OT   = Overshooting Top
#AACP = Above Anvil Cirrus Plume (not available for SEVIRI)
#BOTH = OT and AACP done simultaneously. Optimal params and model input combinations used by default (not available for SEVIRI)
severe_storm_signature = BOTH


#10)
#Would you like to be able to seamlessly transition between previously identified (y/n)?
#best day and night models?
#Best OT model for daytime is TROPDIFF.
#Best AACP model is IR+DIRTYIRDIFF.
#Best BOTH model runs best TROPDIFF for OTs and then best IR+DIRTYIRDIFF for AACPs
optimal_params = y


#11)
#If optimal_params is n or no, what model inputs would you like to use? Keep in mind,
#visible data is not available during night time hours and thus, no model
#results files will be created at night if using VIS, snowice, or cirrus.
#Best OT model for daytime is IR+VIS
#Best OT model at night is IR.
#Best AACP model for daytime is IR+DIRTYIRDIFF
#Best AACP model at night is IR+DIRTYIRDIFF.
#(ex. IR, TROPDIFF, IR+VIS, IR+GLM, IR+WVIRDIFF, IR+VIS+GLM, IR+TROPDIFF,
#IR+CIRRUS, IR+SNOWICE, IR+DIRTYIRDIFF, VIS+TROPDIFF, GLM+TROPDIFF, IR+VIS+TROPDIFF,
#VIS+GLM+TROPDIFF, TROPDIFF+DIRTYIRDIFF, IR+VIS+DIRTYIRDIFF, VIS+TROPDIFF+DIRTYIRDIFF)
#DEFAULT = IR
model_inputs = IR


#12)
#If optimal_params is n or no, what model type would you like to run? Keep in mind,
#visible data will is not available during night time hours and thus, no model
#results files will be created.
#Mutiresunet model has been found to be the best model for both OTs and AACPs
#(ex. multiresunet, unet, attentionunet)
#NOTE: mutltiresunet is the only model type available for AACP model runs
model_type = multiresunet


#13)
#Would you like to map the model results on top of IR/VIS sandwhich images for each
#individual scene (y/n)?
#NOTE: time to plot may cause model run to be slower.
plot_data = n


#14)
#If plot_data is n or no, ignore the remaining 2 prompts.
#If plot_data is y or yes, would you like to use the previously identified best
```

```
#likelihood score threshold for your above choices (y/n)?
optimal_likelihood_response = y


#15)
#If optimal_likelihood_response is set to y or yes, ignore this.
#If optimal_likelihood_response is set to n or no, what thresholds would you like to use
#for creating the plots and identifying objects in post-processing?
#optimal_likelihood_score must be between 0 and 1.
optimal_likelihood_score = 0.4

#16)
#If severe_storm_signature is set to AACP, ignore this.
#If severe_storm_signature is set to OT, percent_omit removes the coldest and warmest X%
#when calculating the mean anvil brightness temperatures (BTDs) in post-processing.
#Default is 20 which means 20% of the warmest and coldest anvil pixel temperatures are
#removed in order to prevent the contribution of noise to the OT IR-anvil BTD calculation.
#percent_omit must be between 0 and 100.
percent_omit = 20
```

3.4 *Optimal Models*

See Table 2 for the model types and inputs along with the optimal model thresholds for each model type allowed.

3.4.1 *OT Optimal Model From Testing*

During the day and night, the OT detection model that performed best during our tests was the TROPDIFF MultiResUnet model. Table 2 shows the optimal threshold for this model is 0.35. See Section 4.4 for more details regarding optimal model thresholds. Users could go down to 0.2 and we believe this would accurately reflect many updraft regions. We chose 0.35, however, because those detections should be more often associated with severe weather. Many storms produce OTs but not every OT-producing storm is associated with severe weather. By choosing 0.35, we believe that these detection locations are more likely to be associated with severe weather events than 0.2.

SEVIRI OT detections also the MultiResNet model with TROPDIFF as the only input. The optimal threshold to use was found to be 0.35 during brief testing phase. Additional testing could be required as this threshold can be conservative in some cases relative to MTG, however, this is also to be expected as MTG has a much higher resolution and thus is not as washed out as the images from SEVIRI.

3.4.2 *AACP Optimal Model From Testing*

During the day and night, the AACP detection model that performed best during our tests was the IR+DIRTYIRDIFF MultiResUnet model. Table 2 shows the optimal threshold for this model is 0.45, however, with the VERSION 4.0.0+ release we are now using an optimal threshold of 0.3. The reason we are using this lower threshold now is because VERSION 4.0.0 added post-processing constraints that significantly reduce the number of false alarms thus allowing us to use a lower threshold where the faulty detections will be removed in post-processing. See Section 4.4 for more details regarding optimal model thresholds.

3.4.3 *Version 2 Software and Older*

Previously, we used IR BTs as our optimal models but found that they saturated when the brightness temperatures were very cold ($<$ 190 K). Instead of using BTs, we now will use tropopause-relative temperature differences as the default. This was how the section was previously written. OTs:

During the day, the OT detection model that performed best during our tests was the IR+VIS MultiResUnet model. Table 2 shows the optimal threshold for this model is 0.25. See Section 4.4 for more details regarding optimal model thresholds.

During the night, the OT detection model that performed best during our tests was the IR MultiResUnet model. Table 2 shows the optimal threshold for this model is 0.40. See Section 4.4 for more details regarding optimal model thresholds.

3.5 *Model Checkpoint Files*

The best model checkpoint files are included during the install process. These files were output during our model training from runs that performed well in tests. They are located in the 'data/model_checkpoints/' directory of the install. These files contain the information that the model needs in order to make detections for AACPs or OTs.

3.6 *Script Breakdown*

The following shows the order of operations for the software and the scripts that are called to carry that out.

1. `run_all_plume_updraft_model_predict.py` is the MAIN function. This function prompts the user with questions regarding what parameters they want to use to run the model and the dates/region to run the model over. A configuration file can also be loaded into this function which answers the questions for the user ahead of time.

2. (a) `run_tf_N_channel_plume_updraft_day_predict` takes model input from any channel combination and calls all of the subroutines to pre-process and run the model. This new function effectively replaces run_tf_1_channel_plume_updraft_day_predict.py, run_tf_2_channel_plume_updraft_day_predict.py, and run_tf_3_channel_plume_updraft_day_predict.py.

   (b) `run_tf_1_channel_plume_updraft_day_predict` takes 1 model input channel (typically IR only) and calls all of the subroutines to pre-process and run the model.

   (c) `run_tf_2_channel_plume_updraft_day_predict` takes 2 model input channels (IR+VIS or IR+GLM) and calls all of the subroutines to pre-process and run the model.

   (d) `run_tf_3_channel_plume_updraft_day_predict` takes 3 model input channels (IR+VIS+GLM) and calls all of the subroutines to pre-process and run the model.

3. (a) `run_download_goes_ir_vis_l1b_glm_l2_data` used in real-time model runs. Downloads the latest GOES ABI and GLM data from Google Cloud's public data storage. See Section 5.2 for more details.

   (b) `run_download_goes_ir_vis_l1b_glm_l2_data_parallel` used in archived model runs. This model uses parallel threads in order to speed up GOES ABI and GLM data downloads from Google Cloud's public data storage. See Section 5.2 for more details.

   (c) `run_download_mtg_fci_data` used in real-time model runs that use MTG data. Downloads the latest MTG data from EUMETSAT public data storage. A EUMETSAT account will be needed to download these files. See Section 5.2 for more details.

(d) `run_download_mtg_fci_data_parallel` used in archived model runs that use MTG data. This model uses parallel threads in order to speed up MTG data downloads from EUMETSAT. A EUMETSAT account will be needed to download these files. See Section 5.2 for more details.

(c) `run_download_seviri_iodc_data` used in real-time model runs that use SEVIRI data. Downloads the latest SEVIRI data from EUMETSAT public data storage. A EUMETSAT account will be needed to download these files. See Section 5.2 for more details.

(d) `run_download_seviri_iodc_data_parallel` used in archived model runs that use SEVIRI data. This model uses parallel threads in order to speed up SEVIRI data downloads from EUMETSAT. A EUMETSAT account will be needed to download these files. See Section 5.2 for more details.

4. (a) `run_create_image_from_three_modalities` used in real-time model runs. If the user specifies they want output maps, this function will open a new Thread to start creating the image in the background while the rest of the model process runs. Once the model is finished running and the output file is available, the parallel Thread will plot the model results onto an IR/VIS sandwich image. Creates "combined" netCDF file of the data. See Section 6 for more details.

(b) `run_create_image_from_three_modalities2` used in archived model runs. This model uses parallel threads in order to speed up archived runs. This function interpolates IR and GLM data onto the VIS data grid to create "combined" netCDF file of the data. See Section 6 for more details.

(c) `run_mtg_create_image_from_three_modalities` Same as (a) but for MTG.

(d) `run_mtg_create_image_from_three_modalities2` Same as (b) but for MTG.

(e) `run_seviri_create_image_from_three_modalities` Same as (a) and (c) but for SEVIRI.

(f) `run_seviri_create_image_from_three_modalities2` Same as (b) and (d) but for SEVIRI.

5. (a) `create_vis_ir_numpy_arrays_from_netcdf_files2` creates IR, VIS, TROPDIFF, GLM, etc. numpy files for a specific date and satellite scan sectors. The numpy arrays are stored (1, number of latitude pixels, number of longitude pixels). This function uses the combined netCDF files (created from combine_ir_glm_vis) to normalize the IR, VIS, and GLM data (0-1) which is then output to numpy arrays that are loaded as input into the model. See Section 7.1 for more details. **IMPORTANT NOTE!!!** IR data values are set to -1 in regions that are not able to be scanned by the satellite. These regions come about due to the gridding process for CONUS and FULL disk scans. They are set to -1 in order to distinguish that there should never be any model detection at those points. When running the model, we can easily find those points and make sure the results likelihoods are set to 0 (not an OT or AACP).

   (b) `mtg_create_vis_ir_numpy_arrays_from_netcdf_files2` same as (a) but for MTG.

   (c) `seviri_create_vis_ir_numpy_arrays_from_netcdf_files2` same as (a) but for SEVIRI.

6. (a) `tf_N_channel_plume_updraft_day_predict` takes model input channels and runs the specified model. Writes model output to numpy file and also opens the combined netCDF file and writes model results there too. This new function effectively replaces tf_1_channel_plume_updraft_day_predict.py, tf_2_channel_plume_updraft_day_predict.py, and tf_3_channel_plume_updraft_day_predict.py.

   (b) `tf_1_channel_plume_updraft_day_predict` takes 1 model input channel and runs the specified model. Writes model output to numpy file and also opens the combined netCDF file and writes model results there too.

   (c) `tf_2_channel_plume_updraft_day_predict` takes 2 model input channels and runs the specified model. Writes model output to numpy file and also opens the combined netCDF file and writes model results there too.

   (d) `tf_3_channel_plume_updraft_day_predict` takes 3 model input channels and runs the specified model. Writes model output to numpy file and also opens the combined netCDF file and writes model results there too.

7. `write_plot_model_result_predictions` is used in archived runs to map the model results for each individual scene as well as a time aggregated map of the model results, SPC reports, and minimum brightness temperatures.

8. `run_write_severe_storm_post_processing` completes post-processing steps. This function adds tropopause to combined netCDF files if not already available there. This function uses the OT/AACP likelihood thresholds to combine detections into OT and AACP objects. Objects are denoted by an OT/AACP ID number variable. It is important to note that tracking of OT and AACP objects are not done and the OT/AACP ID numbers in one file do not correspond to ID numbers from a previous or future file. Some automated quality controlling is done prior to identifying OT or AACP objects.

3.7 *Model Input/Output Files and Locations*

The software creates files that are used as input into the model and also outputs multiple files and, optionally, an image file. This list of input and output files as well as their default locations are listed below.

1. If downloading files, the user can specify the download directory location, however, the default location is '../../../data/goes-data/'. The subdirectory is then the 4 digit year + 2 digit month + 2 digit day. For ESSL MTG and SEVIRI data users, by setting the keyword –essl, the subdirectory is 4 digit year + '/' + 2 digit month + '/' + 2 digit day. This setup makes it so ESSL users, who are already downloading the data on their own, to not make any changes and have the software fit in their framework. IMPORTANT NOTE: The essl setup is only available for MTG and SEVIRI.

2. The first file created is termed in this document as the combined netCDF file. It is created by the combine_ir_glm_vis function, as described in Section 6. An example file header is provided in Section 13.4.1. This file contains satellite information which can be used for mapping the data and model input variables. This file will also be rewritten later to add the contents of the model detections. These files can be read into the McIDAS-V software for visualization. The files are stored in '../../../data/goes-data/combined_nc_dir/'. The subdirectory is then the 4 digit year + 2 digit month + 2 digit day.

3. Numpy files for each model input variable are created for a single time/scan. Depending on the model run chosen, numpy arrays for IR, VIS, and GLM are used as model inputs into the the machine learning algorithm. All of the numpy arrays contain normalized values ranging between 0 and 1. These files are stored in '../../../data/labelled/'. DEFAULT is to delete this files after use if the user does not wish to plot the data using the routines provided. This substantially saves on disk space. See Section 7 for more details.

4. csv file containing IR/VIS/GLM file names with dates and times. This is created in order to keep organized which numpy file goes with which date and raw data file. These files are stored in '../../../data/labelled/'. See Section 7 for more details.

5. DEFAULT is to no longer write these files, starting with Version 2. To save on local memory, the model results will only be written to the netCDF files....Numpy model results file stores the model likelihoods of a detection for each point in the domain. Values range from 0 to 1, with values closer to 1 implying the model is more confident in a detection. These root directory to these files is '../../../data/aacp_results/'. The subdirectories to the file are then the model type (day_night_optimal, IR, IR+GLM, etc.), what the model was trying to detect (updraft or plume), whether it was a real-time or not real-time model, the date that the model was run, the date of interest, and the scan sector.

6. As mentioned in item (2) in this list, the model results are also output to the combined netCDF file. The combined netCDF is opened and a new variable containing the model output, similar to that seen in the numpy output files is written. Some example variable names are ir_ot, ir_glm_ot, ir_glm_aacp, and ir_vis_glm_aacp. This data was written to the combined netCDF files in order to provide an easier user experience and also be able to visualize the results in the McIDAS-V software.

4 *Model Characteristics*

4.1 *Model Training*

For training the model, we used 1-minute mesoscale domain sectors (M1 or M2) GOES-16 data. Table 1 shows the dates and mesoscale regions used. The variable 'Case Classifier' corresponds

to whether or not the date was input into the model as a training (train) or validation (val) dataset. The independent model testing (test) dataset was not input into the model at any time.

Table 1: Dates and times of convective events included in this study.

Date	Time Range (UTC)	Mesoscale Sector	Case Classifier
30 April 2019	1800-2359	1	train
01 May 2019	0000-0043	1	train
05-06 May 2019	2200-0055	1	train
05-06 May 2019	1930-0102	2	train
07-08 May 2019	1800-0100	1	train
17-18 May 2019	1700-0122	2	val
20-21 May 2019	1800-0104	1	train
26-27 May 2019	1800-0056	2	test
13-14 May 2020	1800-0105	2	train

4.2 *Model Types*

See Table 2 for the model types and inputs allowed for each model type.

4.2.1 *OTs*

For detecting OTs, we set up the software to handle a variety of satellite inputs and model types. Most model input combinations only are available for MultiResUnet. Model inputs for OTs available include: IR only, TROPDIFF only, IR+VIS, IR+GLM, IR+WVIRDIFF, IR+VIS+GLM, IR+TROPDIFF, IR+CIRRUS, IR+SNOWICE, IR+DIRTYIRDIFF, VIS+TROPDIFF, GLM+TROPDIFF, IR+VIS+TROPDIFF, VIS+GLM+TROPDIFF, TROPDIFF+DIRTYIRDIFF, IR+VIS+DIRTYIRDIFF, and VIS+TROPDIFF+DIRTYIRDIFF. TROPDIFF stands for 10.3 micron IR BT - GFS Tropopause temperature, WVIRDIFF stands for 6.2 micron - 10.3 micron channel difference, and DIRTYIRDIFF stands for 12.3 micron - 10.3 micron channel difference. For each of these input combinations we tested 3 distinct model architectures: unet, multiresunet, and attentionunet. Since VIS data are not

available at night, we found optimal models for daytime and nighttime.

During the day and night, the OT detection model that provides the most consistent detections across region, season, and time of day during our testing was the TROPDIFF multiresunet model.

4.2.2 *AACPs*

For detecting AACPs, we set up the software to handle a variety of satellite inputs but only 1 model type (MultiResUnet) is available. Model inputs for AACPs available include: IR only, TROPDIFF only, IR+VIS, IR+GLM, IR+WVIRDIFF, IR+VIS+GLM, IR+TROPDIFF, IR+CIRRUS, IR+SNOWICE, IR+DIRTYIRDIFF, VIS+TROPDIFF, GLM+TROPDIFF, IR+VIS+TROPDIFF, VIS+GLM+TROPDIFF, TROPDIFF+DIRTYIRDIFF, IR+VIS+DIRTYIRDIFF, and VIS+TROPDIFF+DIRTYIRDIFF. TROPDIFF stands for 10.3 micron IR BT - GFS Tropopause temperature, WVIRDIFF stands for 6.2 micron - 10.3 micron channel difference, and DIRTYIRDIFF stands for 12.3 micron - 10.3 micron channel difference.

During the day and night, the AACP detection model that performed best during our tests was the IR+DIRTYIRDIFF multiresunet model.

4.3 *Allowed Data Ranges*

For more details on allowed data ranges, refer to create_vis_ir_numpy_arrays_from_netcdf_files2.py or Section 8.2 of this document.

IR BT (10.3μm) range is 180 K to 230 K. VIS reflectance (0.64μm) range is 0 to 1. GLM flash extent density range is 0 to 20. WVIRDIFF (6.2μm - 10.3μm) range is -20 K to 10 K. New TROPDIFF (10.3μm - GFS TropT) range is -12 K to 25 K. Old TROPDIFF (10.3μm - GFS TropT) range is -15 K to 20 K. SNOWICE (1.6μm) range is 0 to 1. CIRRUS (1.37μm) range is 0 to 1. DIRTYIRDIFF (12.3μm - 10.3μm) range is -1 to 2.

4.4 *Optimal Model Likelihood Scores & Thresholds*

For each pixel in the domain, the model outputs the 'confidence' that a pixel is part of the desired severe storm signature. These scores range from 0, not confident, to 1, very confident. Higher scores indicate a higher likelihood that the object you are trying to detect is in fact that object. If you are trying to detect OTs, for instance, a pixel with a score of 0.8 indicates that that pixel is

very likely part of an OT. Using Receiver Operating Characteristic (ROC) curves and Intersection Over Union (IOU) statistical metrics in test cases independent from model training, we have found thresholds that we believe work best for limiting the number of false detections while still identifying the vast majority of severe storm signature events. Now, it is important to note that these thresholds may not be best for each individual case and thus need to be adjusted based on the case the user is reviewing. While running the model, the user has the option to change the thresholds to suit their needs and findings. The optimal thresholds found for each model type and inputs at the VIS resolution of 0.5 km are shown in Table 2. IR resolution (2 km) optimal model thresholds are shown in Table 3.

We have one model for SEVIRI OT detections that uses TROPDIFF, MultiResUnet, and the optimal threshold is 0.35.

5 *Model Input Data*

5.1 *Model Satellite Data Inputs*

Raw Level 1b GOES, GLM data, and GFS tropopause temperatures are used as the primary satellite inputs for training the deep learning models. The ABI is featured on the latest generation of NOAA/NASA geostationary satellites, including GOES-16, -17, -18, and -19 (Schmit et al., 2017, 2018). The ABI samples 16 spectral bands at 0.5-2 km horizontal resolution every 1 min for mesoscale domains, 5 mins for CONUS, and 10 mins for full disk scans. Mesoscale domains are fixed areas, each $\sim$1000 km^2, chosen to provide detailed observations for areas of interest like severe storms. Raw Level 1b GOES-16 channel files used as model input includes the 0.64 μm (VIS band), 1.37 μm (CIRRUS band), 1.6 μm (SNOWICE band), 10.3 μm (IR band), and 12.3μm (dirty IR band). The IR band is interpolated to the VIS band resolution when both inputs are used together. Otherwise, the model software runs using the native IR data resolution. We have also set up the software to handle MTG-1 satellite channels which are available every 10 mins. We have also set up the software to handle MSG1, MSG2, MSG3, MSG4, SEVIRI, and SEVIRI-IODC satellite channels which are available every 15 mins.

The flash extent density product measured by GLM can also be used as input into the deep learning model. Flash extent density indicates the number of flashes (both cloud-to-ground and

Table 2: Model types and optimal thresholds found from independent test cases for data on VIS 0.5 km resolution.

Severe Storm Signature	Model Type	Model Inputs	Optimal Model Threshold
OT	MultiResUnet	IR	0.40
OT	MultiResUnet	TROPDIFF	0.35
OT	MultiResUnet	IR+VIS	0.25
OT	MultiResUnet	IR+GLM	0.65
OT	MultiResUnet	IR+WVIRDIFF	0.30
OT	MultiResUnet	IR+SNOWICE	0.60
OT	MultiResUnet	IR+CIRRUS	0.50
OT	MultiResUnet	IR+DIRTYIRDIFF	0.45
OT	MultiResUnet	IR+TROPDIFF	0.25
OT	MultiResUnet	VIS+TROPDIFF	0.15
OT	MultiResUnet	TROPDIFF+GLM	0.55
OT	MultiResUnet	IR+VIS+GLM	0.40
OT	MultiResUnet	IR+VIS+TROPDIFF	0.15
OT	MultiResUnet	VIS+TROPDIFF+GLM	0.40
OT	MultiResUnet	IR+VIS+DIRTYIRDIFF	0.30
OT	MultiResUnet	VIS+TROPDIFF+DIRTYIRDIFF	0.45
OT	MultiResUnet	TROPDIFF+DIRTYIRDIFF	0.45
OT	Unet	IR	0.45
OT	Unet	IR+GLM	0.45
OT	Unet	IR+VIS	0.35
OT	Unet	IR+WVIRDIFF	0.45
OT	Unet	IR+VIS+GLM	0.45
OT	AttentionUnet	IR+VIS	0.65
OT	AttentionUnet	IR+VIS+GLM	0.20
AACP	MultiResUnet	IR	0.75
AACP	MultiResUnet	TROPDIFF	0.80
AACP	MultiResUnet	IR+VIS	0.50
AACP	MultiResUnet	IR+GLM	0.20
AACP	MultiResUnet	IR+WVIRDIFF	0.80
AACP	MultiResUnet	IR+SNOWICE	0.25
AACP	MultiResUnet	IR+CIRRUS	0.60
AACP	MultiResUnet	IR+DIRTYIRDIFF	0.50
AACP	MultiResUnet	IR+TROPDIFF	0.80
AACP	MultiResUnet	VIS+TROPDIFF	0.70
AACP	MultiResUnet	TROPDIFF+GLM	0.55
AACP	MultiResUnet	IR+VIS+GLM	0.35
AACP	MultiResUnet	IR+VIS+TROPDIFF	0.70
AACP	MultiResUnet	VIS+TROPDIFF+GLM	0.30
AACP	MultiResUnet	IR+VIS+DIRTYIRDIFF	0.30
AACP	MultiResUnet	VIS+TROPDIFF+DIRTYIRDIFF	0.25
AACP	MultiResUnet	TROPDIFF+DIRTYIRDIFF	0.75

Table 3: Model types and optimal thresholds found from independent test cases on IR 2.0 km resolution.

Severe Storm Signature	Model Type	Model Inputs	Optimal Model Threshold
OT	MultiResUnet	IR	0.20
OT	MultiResUnet	TROPDIFF	0.40
OT	MultiResUnet	IR+GLM	0.25
OT	MultiResUnet	IR+WVIRDIFF	0.55
OT	MultiResUnet	IR+DIRTYIRDIFF	0.25
OT	MultiResUnet	IR+TROPDIFF	0.40
OT	MultiResUnet	TROPDIFF+GLM	0.55
OT	MultiResUnet	TROPDIFF+DIRTYIRDIFF	0.45
AACP	MultiResUnet	IR	0.30
AACP	MultiResUnet	TROPDIFF	0.60
AACP	MultiResUnet	IR+GLM	0.30
AACP	MultiResUnet	IR+WVIRDIFF	0.55
AACP	MultiResUnet	IR+DIRTYIRDIFF	0.40
AACP	MultiResUnet	IR+TROPDIFF	0.20
AACP	MultiResUnet	TROPDIFF+GLM	0.50
AACP	MultiResUnet	TROPDIFF+DIRTYIRDIFF	0.15

cloud-to-cloud) that occur within a grid cell over a given time period. The horizontal resolution of the flash extent density data are 8 km with new GLM data files available every 20 seconds. GLM data are gridded using the glmtools Python library provided in Bruning et al. (2019). Only GLM files within ±2.5 minutes of GOES IR/VIS start scan times are gridded and considered co-located. GLM data are also interpolated to the VIS data resolution of 0.5 km prior to being input into the model. Lightning is not yet available for MTG satellite imagery runs.

In addition to satellite imagery, the Global Forecast System (GFS) model tropopause temperature product was tested as input to the machine learning models. GFS was chosen due to its production of near real-time analysis files. GFS grib files are downloaded from the NCAR Research Data Archive. The GFS lapse-rate tropopause temperature product (TropT), available 6-hourly on a 0.25° x 0.25° longitude-latitude grid (National Centers for Environmental Prediction, National Weather Service, NOAA, U.S. Department of Commerce, 2015), is linearly interpolated in time and space from the GFS grid to the GOES ABI pixels. Occasionally, reanalysis tropopause temperature fields can show unrealistic spatial variability. At one location, the reanalysis may accurately capture the primary tropopause while nearby it may incorrectly label the secondary tropopause as the primary. This is most common near jet streams (Khlopenkov et al., 2021). To mitigate potential artifacts, tropopause temperature outliers are identified and replaced using spatial filtering methods described in Khlopenkov et al. (2021). For our purposes, we used a 10 grid box radius for the spatial filtering.

5.2 *Data Download Sources*

It is recommended that users download data from the Google Cloud Platform in order to get the full usage out of the software, but it is not required to run the software. Additional resources are included below. As a reminder, our software uses the L1b GOES data products. In addition, our software uses EUMETSAT data services in order to download MTG and SEVIRI satellite data.

5.2.1 *Google Cloud*

GOES and GLM data can be downloaded from Google Cloud (see Section 9 for more information about the Google Cloud Platform). All satellite imagery used in this study were obtained from the GCP (Google). Data are available in Google's public storage buckets within a

minute of the satellite scan completion. Software functions that do the downloading are named run_download_goes_ir_vis_l1b_glm_l2_data.py and run_download_goes_ir_vis_l1b_glm_l2_data_parallel.py functions. Using Google Cloud, data can be dowloaded in 'real-time' (1 VIS file, 1 IR file, and 15 GLM files) closest to the time of day the script is run. 15 GLM files are downloaded because data files are created every 20 seconds and we want enough data to use with the 1-minute IR and VIS data. See Section 9 for Google Cloud setup instructions.

GFS tropopause temperature data when used in real-time is also downloaded from Google Cloud. If tropopause temperatures in post-processing, then the data are downloaded from data.rda.ucar.

5.2.2 *Additional GOES Resources*

The software is setup to be run if you have locally stored GOES files and do not want to download from the GCP. If you would prefer to download the data yourself to store and not use Google Cloud storage, please refer to page 14 of NOAA Website Documentation. I have provided this link because I think they do a better job of linking to sites and explaining the process than I would here. This page of the document refers to NOAA's CLASS data repository. As a reminder, our software uses the L1b GOES data products.

A similar but easier website to navigate might be NCEI's Archive Information Request System (AIRS). No account is needed for this site (AIRS Website). Description of downloading GOES data from AIRS is also on page 14 of the pdf link sent above. Both NOAA's class system and AIRS will allow you to download GOES data using FTP.

5.2.3 *EUMETSAT Data Services*

The previous subsections discuss how to download GOES satellite imagery. Users who wish to use MTG or SEVIRI satellite imagery will need to follow the steps outlined in Section 13.3. Data are available on this portal for both real-time and archived applications. Software functions that do the downloading are named run_download_mtg_fci_data.py, run_download_mtg_fci_data_parallel.py, run_download_sev
run_download_seviri_iodc_data_parallel.py functions.

6 *Output netCDF Files*

6.1 *Overview*

The pipeline necessary to create a layered image and combined netCDF file has four parts: 1) Main run script, 2) GLM data gridding (optional), 3) Combining IR, VIS, and GLM data into netCDF file, and 4) image creation. These functions are named run_create_image_from_three_modalities (or run_create_image_from_three_modalities2 for parallel jobs in archived model runs), glm_gridder, combine_ir_glm_vis, and img_from_three_modalities2, respectively. The main function (1) runs the other three subroutines. These "combined" netCDF files can be loaded into the McIDAS-V software.

6.2 *Script Breakdown*

1. `run_create_image_from_three_modalities` is the MAIN PROGRAM. Another program that can act as the main program is `run_create_image_from_three_modalities2`. This program runs the netCDF file creations in parallel threads which is useful for archived model runs. The programs import and call all of the following functions. User specifies the input directory (inroot) to the GOES IR/VIS and GLM raw data directories and the function creates the GLM gridded file and loops over all of the GOES IR and VIS files (in alphabetical order.). Optional keywords for GLM data included in netCDF file creation (no_write_glm keyword). The default functionality is to not write or plot the GLM data (saves time and space and not needed for IR/VIS sandwich images). Optional keywords for VIS data included in netCDF file creation (no_write_vis keyword). Setting the keyword to True allows the job to be run on the native IR resolution which is lower than VIS. Ultimately, this saves time and space. The default functionality is to not write or plot the VIS data if VIS data are not required.

2. `glm_gridder` grids and reformats the GLM lightning flash data and writes output to a netCDF file. Grids ONLY files within $\pm$2.5 minutes of GOES IR/VIS start scan time as default. The 'twindow' keyword can specify time window of files surrounding GOES IR/VIS start scan time. Example file ncdump header is provided in Section 13.4.1. This file is read

in combine_ir_glm_vis.py.

3. `combine_ir_glm_vis` reads in the three data types and finds the subset of GLM data that matches the IR data. Function also converts the visible radiance to reflectance and then normalizes reflectance by the solar zenith angle (at every lat/lon pixel), IR radiance to brightness temperature, and calculates the latitude and longitude of the selected viewpoint. The latitude and longitude are then written to a netCDF file along with the data from the three modalities. IR data are written in terms of brightness temperature, VIS data are written in terms of reflectance normalized by the solar zenith angle, solar zenith angle in degrees, and GLM data are written in terms of flash extent density. The ncdump header information of an example file is provided in Section 13.4.2. The user can choose whether or not they want to include GLM data or VIS data in the output netCDF file.

4. `combine_ir_glm_vis_parallel` is the same as combine_ir_glm_vis.py but uses caching of domain sector information to allow for much faster run times in archived mode.

5. `glm_data_processing` contains utility functions that are used to grid/project the GOES data and convert it into desired units.

6. `img_from_three_modalities2` creates a composite PNG image from the combined netCDF file. If VIS data are not available or the zenith angle in the middle of the image scene exceeds 85°, only the IR data are plotted. VIS data are plotted using greyscale while IR data are plotted using a rainbow color table. A colorblind version of the rainbow color table created using Google AI and named turbo_colormap is also available and is set as the function's default. Figures 1 and 2 provide an example on IR/VIS sandwich image and the color bar, respectively.

6.3 *Path Setup*

The default file path setup is listed below. The keywords in run_create_image_from_three_modalities will do most of the work and decide what directory to send the images based upon the keywords that are set.

1. GLM, IR, and VIS input files = '../../../data/goes-data/20200513/'. The files for each GOES file type is within subdirectories 'glm/', 'ir/', and 'vis/', respectively.

2. GLM outfile directory = '../../../data/goes-data/out_dir/'. Location of re-gridded GLM data file.

3. Combined IR, GLM, and VIS directory file directory = '../../../data/goes-data/combined_nc_dir/'.

6.4 *Removed Data*

IR BT $<$ 163 K or NaN are set to 500 K. This removes incomplete or invalid data and makes sure that those points will not be used by the model. Bad pixels will be sporadic, often on domain edges or in a bad scan line, and not possessing a spatial configuration that should look like an OT or AACP to a deep learning model. Often times such bad pixels/lines have anomalously cold temperatures. With that said, it would probably be best to have it register as a 0 weight in the scaled brightness temperature so that it is disregarded.

ONLY GLM flash extent density within $\pm$2.5 minutes of the GOES IR/VIS scan are used and written to the combined netCDF files.

Visible, snowice, and cirrus data are not available if the solar zenith angle in 5% or more of the domain exceeds 85°. In this case, ONLY IR and WV data channels are used. This prevents retention of data with artificially high reflectance values as a result of solar zenith angle transitioning from daytime to nighttime.

7 *Creating Numpy Arrays for Model Input*

7.1 *Overview*

Numpy arrays for IR, VIS, TROPDIFF, DIRTYIRDIFF, CIRRUS, SNOWICE, WVIRDIFF, and GLM are used as model inputs into the the machine learning algorithm. Numpy arrays are created for a single time. All of the numpy arrays contain normalized values ranging between 0 and 1. The scripts necessary to create a numpy array has 2 parts: 1) Create model input variable numpy arrays and 2) Create corresponding csv file that gives how the numpy files are organized. The script used is create_vis_ir_numpy_arrays_from_netcdf_files2.py.

7.2 *Script Breakdown*

This first list shows the script breakdown for creating numpy mask files.

1. `create_vis_ir_numpy_arrays_from_netcdf_files2` and `mtg_create_vis_ir_numpy_arra` are the MAIN functions functions that create IR, VIS, TROPDIFF, GLM, etc. numpy files for a specific date and scan sector for GOES and MTG satellite data, respectively. ata are both interpolated onto the VIS data grid, if they were not already. This function uses the combined netCDF files (created from combine_ir_glm_vis) to normalize the IR, VIS, and GLM which is then output to numpy arrays. The numpy arrays are stored as with dimensions (1, number of latitude pixels, number of longitude pixels).

2. `fetch_convert_vis` is a function that is called by create_vis_ir_numpy_arrays_from_netcdf_files2 and normalizes the reflectance data to range from 0 to 1. This function returns the normalized reflectance array and whether the GOES scan is considered night or day (if $> 5\%$ of pixels in domain $> 85^{\circ}$ yields night). This was done in order to avoid poor detections in high solar zenith angle regions.

3. `fetch_convert_ir` is a function that is called by create_vis_ir_numpy_arrays_from_netcdf_files2 and normalizes the brightness temperatures by user specified minimum and maximum values. The default min and max values are 195 K and 225 K, respectively. Brightness temperature measurements closer to the min value are nearer to 1 when normalized. IMPORTANT NOTE!!! IR data values are set to -1 in regions that are not able to be scanned by the satellite. These regions come about due to the gridding process for CONUS and FULL disk scans. They are set to -1 in order to distinguish that there should never be any model detection at those points. When running the model, we can easily find those points and make sure the results likelihoods are set to 0 (not an OT or AACP).

4. `fetch_convert_trop` is a function that is called by create_vis_ir_numpy_arrays_from_netcdf_files2 and normalizes the IR BT - TropT difference data to range from 0 to 1. Previous methods used a linear weighting function scheme but the new method scheme uses a cosine weighting function as shown by the green line in Figure 3. The default min and max values used -15 and 20, respectively, and are now -12 and 25. We saw that it was rare for IR BTs to be more than

12 K colder than the tropopause and if they were that much colder it was very likely a valid OT. In addition, by extending the maximum value to 25 K, we allow for detections warmer than tropopause to be considered OTs because not every OT is colder than the tropopause. Lastly, we decided to use a cosine function because it also yields more temperature gradients across an image and would allow for more detections warmer than tropopause due to those temperature gradients.

5. `fetch_convert_snowice` is a function that is called by create_vis_ir_numpy_arrays_from_netcdf_files2 and normalizes the snow/ice reflectance data to range from 0 to 1. The default min and max values are 0 and 1, respectively.

6. `fetch_convert_cirrus` is a function that is called by create_vis_ir_numpy_arrays_from_netcdf_files2 and normalizes the cirrus reflectance data to range from 0 to 1. The default min and max values are 0 and 1, respectively.

7. `fetch_convert_dirtyirdiff` is a function that is called by create_vis_ir_numpy_arrays_from_netcdf_f and normalizes the dirtyIR - IR difference data to range from 0 to 1. The default min and max values are -1 and 2, respectively.

8. `fetch_convert_irdiff` is a function that is called by create_vis_ir_numpy_arrays_from_netcdf_files2 and normalizes the WV-IR difference data to range from 0 to 1. The default min and max values are -20 and 10, respectively.

9. `fetch_convert_glm` is a function that is called by create_vis_ir_numpy_arrays_from_netcdf_files2 and normalizes flash extent density data by user specified min and max values. The default min and max values are 0 and 20, respectively.

10. `merge_csv_files` is a function that merges ALL csv files created from the previous steps. Output is one big, combined csv file containing label csv files and IR/VIS/GLM files with dates and times.

8 *Post-Processing Model Results*

8.1 *Overview*

Once model runs are complete, the software now provides post-processed data into the output netCDF files. The post-processed products include object identification numbers, the brightness temperature difference between the anvil and the OT, as well as the GFS tropopause temperature. As of 30 July 2025, post-processing now includes an automated quality control process for AACPs and OTs to remove spurious detections. The quality control process for AACPs was added to remove detections far from cold cores as well as remove detections along the edges of convective cells where false positives were identified. For OTs, any OT ID with OT-anvil mean brightness temperature difference $>$ 1 K is removed, suggestive of the OT being the same temperature or warmer than the anvil, is removed from OT identifications. For warmer OTs ($\geq$ 225 K), we require the OT minimum be 6+ K colder than the surrounding anvil to reduce spurious detections. Raw probabilities within the output netCDF file are unchanged but even though they are larger than the optimal likelihood scores, they are removed from consideration.

Once model runs are complete, the software provides post-processed data into the output netCDF files. The post-processed products include object identification numbers, the brightness temperature difference between the anvil and the OT, as well as the GFS tropopause temperature. As of 30 July 2025, post-processing now includes an automated quality control process for AACPs and OTs to remove spurious detections. For OTs, any OT ID with an OT-anvil mean brightness temperature difference $>$ -1 K is removed, suggestive of the OT being the same temperature or warmer than the anvil. For warmer OTs ($\geq$ 225 K), we require the OT minimum be 6+ K colder than the surrounding anvil to reduce spurious detections. Raw probabilities within the output netCDF file are unchanged, but pixels failing these physical checks are removed from the final object identifications. As of 17 April 2026, the quality control process for AACPs was updated to remove even more detections far from convective cores, eliminate false positives along the edges of convective cells, and enforce structural and contextual rules (such as shape density and proximity to detected OTs).

8.2 *Script Breakdown*

This first list shows the script breakdown for creating numpy mask files.

1. `run_write_severe_storm_post_processing` is the MAIN function that appends the output netCDF files. This function calculates brightness temperature differences and extracts object ID numbers.

2. `append_combined_ncdf_with_model_post_processing` is a function that is called by run_write_severe_storm_post_processing that opens the model output netCDF file and appends it with the post-processed data variables.

3. `download_gfs_analysis_files` is a function that uses the request python module to download GFS GRIB analysis files from NCAR.

4. `gfs_nearest_time` is a function that calculate the date and time of the GFS analysis or forecast file nearest to 'date'.

5. `gfs_interpolate_tropT_to_goes_grid` is a function that interpolates tropopause temperatures from GFS onto a satellite data grid.

8.3 *Post-Processing Variables and Calculations*

8.3.1 *Object Identification Numbers*

There are 2 possible object types that the User can specify to the software, OT and AACP. In the configuration file, the User has the option to specify the likelihood threshold around which they would like the data to be post-processed. The User can also not specify this or choose to use the optimal threshold found during testing (see Table 2). Starting with VERSION 4.0.0, the User can also specify BOTH which will run OTs and AACPs. The optimal inputs and thresholds for OTs and AACPs are used when BOTH is specified. In post-processing, the dataset is first cleaned by removing all pixels with output likelihood values less than 0.05. Next, the software uses `scipy.ndimage.label` to identify individual objects containing at least one pixel exceeding the optimal likelihood threshold.

For OTs, the software retains all pixels that are at least 50% of the value of the maximum likelihood score in the object. For AACPs, the boundary threshold is dynamically scaled between 10% and 80% of the object's peak likelihood to better capture faint plume tails while maintaining a tight core.

AACPs are then subjected to rigorous morphological and spatial quality control. The software evaluates the plume's fill ratio, minimum core size, distance to the parent convective cell, and proximity to the nearest OT (`aacp_ot_max_dist`). Additionally, a dilation "ring" test is performed to identify and remove false positives located on the warm edges of the anvil, and size ratio checks are applied to prevent the model from erroneously contouring the entire updraft core instead of a distinct trailing plume. Each pixel that satisfies these conditions within an object is given the same ID number. The ID numbers apply uniquely to each satellite scan and therefore cannot be used to track an object throughout its lifetime. ID numbers apply uniquely to each satellite scan, i.e. ID number 1 in one scan will likely not be the same feature as ID number 1 in the next scan, and therefore cannot be used to track an object throughout its lifetime.

8.3.2 *IR OT-Anvil Brightness Temperature Difference*

IR OT-Anvil Brightness Temperature Difference (BTD) is calculated for OTs only. For each OT object ID, the software identifies the coldest BT in the object. Then, the software searches for anvil pixels within a 30x30 km domain surrounding the coldest pixel in the OT. A pixel is considered to be anvil if it is not part of any OT object. The software the omits the coldest and warmest X% when calculating the mean anvil brightness temperature. The default for X is 20% which means that 20% of the warmest and coldest anvil pixel temperatures are removed in order to prevent the contribution of noise to the OT IR-anvil BTD calculation. This value can be specified by the User in the configuration file. The mean brightness temperature of the remaining anvil pixels is calculated and subtracted from the minimum BT in the OT in order to get IR OT-Anvil BTD.

8.3.3 *Tropopause Temperature*

Tropopause temperatures are calculated from GFS analysis files. GFS analysis files are downloaded from the NCAR data archive and will automatically download analysis times that encompass the date range that the model was run over, whether it was a real-time or archived model

run. The GFS lapse-rate tropopause temperature product (TropT), available 6-hourly on a 0.25° x 0.25° longitude-latitude grid (National Centers for Environmental Prediction, National Weather Service, NOAA, U.S. Department of Commerce, 2015), is linearly interpolated in time and space from the GFS grid to the GOES ABI pixels. Occasionally, reanalysis tropopause temperature fields can show unrealistic spatial variability. At one location, the reanalysis may accurately capture the primary tropopause while nearby it may incorrectly label the secondary tropopause as the primary. This is most common near jet streams (Khlopenkov et al., 2021). To mitigate potential artifacts, tropopause temperature outliers are identified and replaced using spatial filtering methods described in Khlopenkov et al. (2021). In Version 2 and earlier we used a 10 GFS grid box radius for the spatial filtering, however, found this to be insufficient so starting in Version 3 we use 20 GFS grid boxes.. Tropopause temperatures are only included in the output netCDF files for OT object model runs or for runs that used TROPDIFF as model input.

9 *Google Cloud Platform (GCP)*

9.1 *Overview*

IMPORTANT NOTE 1: Google Cloud is not essential for this project. All of the software can be run without installing Google Cloud, however, users that want to run the model in real-time would be advised to install Google Cloud Services such that, data can be downloaded from the cloud. In testing we often found the ucar.rda site not responsive and in Version 3 made Google Cloud the backup location to use for downloading GFS data.

IMPORTANT NOTE 2: In order to use Google Cloud services, the user must have a gmail account. Downloading data from the Google Cloud public storage repository is free with a gmail account.

Raw Level 1 GOES files are downloaded from Google public storage buckets and we often store files that are used as input into the model on our own storage buckets which can be accessed by Google's Virtual Machines (VMs). This project ran the model training, validation, and prediction on Google's VMs. This section will discuss the storage buckets in which the data are stored, the VMs, and code used to writing and reading from storage buckets. All of the scripts, made by John Cooney, are capable of reading and writing the data locally as well as to and from the Google

Cloud Storage Buckets. If the following subsections are not helpful, refer to the code for some examples on how to read and write from the GCP buckets.

9.2 *Installing Google Cloud*

The following steps are used to install Google Cloud on your device. I have included as many details and separated the tasks to be small so that they are as easy to follow as possible. Hopefully, Google does not change how this process.

Step 1: To install Google Cloud, follow the directions on Google Cloud Install. Download the package for your operating system.

Step 2: Follow instructions on the page (steps 2a, 2b, and 2c). Even if it says Optional, I recommend doing those steps. The instructions include unzipping the downloaded file, running the install script (from the root of the folder you extracted in the last step) as described on the website (ex. .'/google-cloud-sdk/install.sh'), and allowing the install to update your PATH in your .bash_profile or .zshrc whatever you are using. Your .bash_profile or .zshrc should look similar to the following after completing these steps:

```
# The next line updates PATH for the Google Cloud SDK.
if [ -f '/Users/user-name/google-cloud-sdk/path.bash.inc' ]; then . '/Users/user-name/google-cloud-sdk/path.bash.inc'; fi

# The next line enables shell command completion for gcloud.
if [ -f '/Users/user-name/google-cloud-sdk/completion.bash.inc' ]; then . '/Users/user-name/google-cloud-sdk/completion.bash.inc'; fi
```

Step 3: Initialize the gcloud CLI by running 'gcloud init'. Running this will give the user prompts they need to answer.

First, the user will be prompted for a configuration name. This configuration name can be anything that the user wants, as long as it is not already being used by another user project. In testing, I chose tmptestproj2-7363823364 because it was very unlikely to have been used by someone else previously. You can choose something that corresponds more closely to your project though.

Second, the user will be prompted for an account to be associated with this configuration (user must have a Gmail account in order for this to work).You can login with new account when prompted.

This will redirect the user to their default web browser.

Click on your gmail account or enter it in and login to the account.

Click 'Allow' at bottom of page to authenticate the Google Cloud CLI.

User will next be prompted to create a project ID.

You can again name this anything you would like as long as it is not being used by another Google Cloud user.

Example: In testing, I chose a random assortment (e.g. tmptestproj-673527384)

Step 4: Once signed in, click select a project (next to 'GoogleCloud text' in left hand corner).

Step 5: Select your project name.

Step 6: Select the 3 horizontal lines to the left of 'GoogleCloud' text.

Step 7: Scroll down to 'IAM & Admin'.

Step 8: Within 'IAM & Admin' menu, Click on 'Service Accounts'.

Step 9: Once at Service Accounts page, Click '+ CREATE SERVICE ACCOUNT' near top of page.

Step 10: Type in a Service Account name. Again, this can be anything you want. Once you have typed in the Service Account name, Click Done. You have now created a service account.

Step 11: Click the 3 vertical dots next to the service account that you just enabled.

Step 12: Scroll down and Click Manage Keys.

Step 13: Click 'ADD KEY' on webpage.

Step 14: Click 'Create new key' (Create a JSON key type). This will download the json file to your local computer.

Step 15: Move the json file that was just downloaded into your google-cloud-sdk directory.

Step 16: Within your .zshrc, .basrc, or whatever it is that you use, type:

export GOOGLE_APPLICATION_CREDENTIALS=path-to-json-file

(e.g. in my file, 'export GOOGLE_APPLICATION_CREDENTIALS=/Users/jack/google-cloud-sdk/tmptestproj-673527384-a2baeabf78a0.json').

Step 17: Open a new Terminal window. All new Terminal windows should now be able to use Google Cloud which will be accessed by the software code.

This should allow you to use gsutil commands. For downloading, you may need to run *gcloud iam service-accounts keys create /Users/user-name/google-cloud-sdk/google-service-account-file.json –iam-account=project-name@appspot.gserviceaccount.com* in your terminal in order to get necessary json file. Put this file in a location that you want and then export it in .zshrc or .bash_profile by adding a line similar to:

```
export GOOGLE_APPLICATION_CREDENTIALS=/Users/user-name/google-cloud-sdk/google-service-account-file.json
```

9.3 *Google Cloud Problems Users May Encounter*

9.3.1 *Local Web Proxy/Firewall Error*

When running the 'glcoud init' command (to initialize the glcoud CLI utilities), some users have reported running into an error message that looks like the following:

```
ERROR: Reachability Check failed.
    httplib2 cannot reach https://accounts.google.com:
[SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed: self signed certificate in certificate chain (_ssl.c:1129)

    httplib2 cannot reach https://cloudresourcemanager.googleapis.com/v1beta1/projects:
[SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed: self signed certificate in certificate chain (_ssl.c:1129)

    httplib2 cannot reach https://www.googleapis.com/auth/cloud-platform:
[SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed: self signed certificate in certificate chain (_ssl.c:1129)

    httplib2 cannot reach https://dl.google.com/dl/cloudsdk/channels/rapid/components-2.json:
[SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed: self signed certificate in certificate chain (_ssl.c:1129)

Network connection problems may be due to proxy or firewall settings.
```

The key step to fixing this issue was doing: gcloud config set core/custom_ca_certs_file $SSL_CERT_FILE. Your system should then populate this environment variable with a specific tls-ca-bundle.pem file on it, allowing the gcloud products to all work.

If the above fix does not work for you, I found some other resources that might be helpful and included them here. One a possible solution to the problem is setting: gcloud config set auth/disable_ssl_validation True. This would disable the ssl validation. Some other links that may be useful are provided here: Stack Overflow Link and Google Cloud Documentation Link. Your resident IT expert may be best to discuss the proxy issue with though as certain user's settings may have different permissions and allowances than others.

9.4 *Script Breakdown*

All of the code for reading and writing to and from the Google Cloud is located within the script *gcs_processing.py*. All of the software code calls the functions in this script. There are multiple functions that will list the files within a particular google storage bucket, read a file on the storage bucket, download a file from a storage bucket, and write a file to a storage bucket. The functions are capable of handling csv files, numpy files, image files, netCDF files, and model checkpoint files. Choose the function that best suits your needs.

NOTE, netCDF and model checkpoint files must be available locally in order to be read. Thus, we must download them from a Google Cloud storage bucket. Other file types can just be loaded into memory directly from the Google Cloud storage buckets so they do not need to be downloaded first.

10 *Software Updates*

10.1 *MAJOR REVISIONS*

10.1.1 *05 June 2023*

Added post-processing function that yields object ID numbers of OTs and AACPs as well as OT IR-anvil brightness temperature differences.

Fixed issue that occurred in full disk and CONUS scanned model runs. The issue occurred at the very edge of the domain where the satellite view is off in to space on the edge of Earth. This issue was fixed prior to release for OTs but AACPs needed to be fixed in a unique way.

10.1.2 *22 June 2023*

Added tropopause temperature to post-processing functionality for OT model runs. This software downloads GFS data from NCAR and then interpolates and smooths the tropopause temperature onto the satellite data grid before writing the output to the netCDF files.

10.1.3 *23 August 2023*

Added Windows .yml file to the git repo. This should allow Windows users an easier time to setup and install the software. Minor changes to path in run_tf_1_channel_plume_updraft_day_predict.py function as well that occurred when re split was being used and the pattern being searched for included path separators.

10.1.4 *6 March 2024 VERSION 2 Release*

VERSION 2 of software!!! LARGE model performance enhancement, particularly with AACP detection. Added A LOT of new model input combinations as possibilities, including TROPDIFF,

DIRTYIRDIFF, SNOWICE, and CIRRUS. Added the new and improved checkpoint files associated with those model runs. Optimal models and thresholds have been updated since the Version 1 release. Changed how we interpolate the GFS data onto the GOES grid. We now follow the methods outlined in Khlopenkov et al. (2021). New version changes how software decides between daytime and nighttime model runs. Previously used the maximum solar zenith angle within the domain and checked if that exceeded 85°, however, there was an issue of only nighttime models being used for CONUS domains due to how the satellite views the Earth and the data are gridded. Thus, now the software checks to see if more than 5% of the pixels in the domain have solar zenith angles that exceed 85°. If they do, then the software acts as if the domain is nighttime and if not the software acts as if it is daytime within the domain. Fixed major issue with looping over post-processing dates and the month or year changed during a real-time or archived model run. New version speeds up post-processing for OTs. Set variables to None after using them in order to clear cached memory. Catch and hide RunTime warnings that the User does not need to worry about.

Software Users prior to 6 March 2024 will need to pull the latest python files from GitHub. This should be very fast. Users MUST also download the new checkpoint files from 'https://science-data.larc.nasa.gov/LaRC-SD-Publications/2023-05-05-001-JWC/data/ML_data.zip'. Once downloaded and unzipped, replace the old model_checkpoints subdirectory with the latest one that was just downloaded. This directory includes all of the improved model detection files as well as checkpoint files for the new input combinations. If you have issues downloading and unzipping the file from the browser, run the run_download_model_chkpoint_files.py script. Users have experienced issues downloading the checkpoint files from the browser and this script has been successful.

10.1.5 *2 May 2024*

Visualization product offered with the python package. This add on product allows for images to be created for both real-time and archived model runs. The script that does the mapping is named img_from_three_modalities2.py. In real-time, images are started in the background while the software is pre-processing and running the ML model on the data. In archived mode, images are created after each day is run. Thus, output images ARE NOT object identifications. Object identifications are made in post-processing. Instead, the output is just OT/AACPs that exceed some likelihood threshold.

NOTE, plotting the data does slow down the software as it takes some time to create the maps. However, real-time run tests have shown the software still easily keeps up with GOES ABI sampling frequency. Thus, everything from downloading the model inputs to creating an output map are completed prior to the next GOES scan is available.

User must install cartopy version 0.20.1 in the svrstormsig conda environment. Try pip install cartopy==0.20.1

10.1.6 *9 January 2025*

issue was identified with the native IR OT detection checkpoint files located in directory path: data/model_checkpoints/ir/updraft_day_model/2023-11-21/multiresunet/chosen_indices/by_date/by_updraft/. We are unsure what caused this checkpoint file corruption but it has since been fixed.

Software Users prior to 9 January 2025 will need to download the latest checkpoint files from 'https://science-data.larc.nasa.gov/LaRC-SD-Publications/2023-05-05-001-JWC/data/ML_data.zip'. Once downloaded and unzipped, replace the old model_checkpoints subdirectory with the latest one that was just downloaded. This directory includes all of the improved model detection files as well as checkpoint files for the new input combinations. If you have issues downloading and unzipping the file from the browser, run the run_download_model_chkpoint_files.py script.

10.1.7 *16 April 2025 VERSION 3 Release*

VERSION 3 of software!!! Major changes include making software more applicable to global applications as well as the additions of MTG-1 and GOES-19 satellites into the software framework. Optimal models and thresholds have been updated since the Version 2 release for TROPDIFF and VIS+TROPDIFF OT models. A new default VIS+TROPDIFF OT model is included with this release. TROPDIFF and VIS+TROPDIFF OT detection model inputs are now the default. Our team decided to change these to the default inputs because IR-only and IR+VIS was not performing up to standards for global situations across various seasons. This change should hopefully improve model detections. We also changed how we interpolate the GFS data onto the GOES grid. Previously, we used 10 GFS grid boxes to smooth the tropopause temperatures but now we use 20. We chose to use additional grid boxes because we noticed that in some cases there were tropopause temperature discontinuities which resulted in OT detections where the tropopause was warmer

even though the underlying satellite imagery did not show an OT. Another change that was added was that in the optimal run settings and not plotting the data, we delete the intermediate numpy files after using them. By deleting these files, Users will save a lot of disk space.

Because of the new satellites, intermediate files for those satellites has been created. Thus, Software Users prior to 16 April 2025 will need to pull the latest python files from GitHub. This should be very fast. Users MUST also download the new checkpoint files from 'https://science-data.larc.nasa.gov/LaRC-SD-Publications/2023-05-05-001-JWC/data/ML_data.zip'. Once downloaded and unzipped, replace the old model_checkpoints subdirectory with the latest one that was just downloaded. This directory includes all of the improved model detection files as well as checkpoint files for the new input combinations. If you have issues downloading and unzipping the file from the browser, run the run_download_model_chkpoint_files.py script. Users have experienced issues downloading the checkpoint files from the browser and this script has been successful.

10.1.8 *30 July 2025 VERSION 3.3.0 Release*

VERSION 3.3.0 of software!!! This update introduces a major revision to the TROPDIFF OT VIS resolution weighting scheme used for model input. Previously, the weighting was applied linearly between -15 K and +20 K relative to the tropopause, but this approach was found to be overly conservative and often failed to detect valid overshooting tops (OTs) that were warmer than the tropopause. In the new version, the weighting scheme uses a cosine function ranging from -12 K to +25 K. This adjustment enhances temperature gradients across a scene, improving the model's ability to detect warmer OTs that are colder than their surrounding anvils. The lower bound was changed from -15 K to -12 K because cases where OT temperatures were more than 12 K colder than the tropopause were rare, but when they did occur, they reliably indicated strong OTs and should receive the maximum weight. The upper limit was extended from +20 K to +25 K to include warmer cases that previously received minimal or no weight. Importantly, the cosine function was found to appropriately exaggerate the gradient without overdoing it; the curve begins to flatten around +15 K, beyond which OTs are unlikely, providing a natural taper in the weighting. Overall, this new scheme improves detection sensitivity while preserving physical realism in the temperature field.

Because of the new TROPDIFF model, intermediate files for those satellites has been cre-

ated. Thus, Software Users prior to 30 July 2025 will need to pull the latest python files from GitHub. This should be very fast. Users MUST also download the new checkpoint files from 'https://science-data.larc.nasa.gov/LaRC-SD-Publications/2023-05-05-001-JWC/data/ML_data.zip'. Once downloaded and unzipped, replace the old model_checkpoints subdirectory with the latest one that was just downloaded. This directory includes all of the improved model detection files as well as checkpoint files for the new input combinations. If you have issues downloading and unzipping the file from the browser, run the run_download_model_chkpoint_files.py script. Users have experienced issues downloading the checkpoint files from the browser and this script has been successful.

Fixed issue where archived mode quit running due to unequal numbers of files in the input and combined netCDF files. New method checks for date strings in order to match files rather than just using the directory itself. This should fix the issue.

10.1.9 *17 April 2026 VERSION 4.0.0 Release*

VERSION 4.0.0 of the software!!! This major update introduces the highly requested ability to run both Overshooting Top (OT) and Above Anvil Cirrus Plume (AACP) models simultaneously in both real-time and archived modes. By specifying 'BOTH' in the configuration file, the software automatically applies the default optimal thresholds and models to detect both features concurrently. To support this, the underlying architecture was streamlined: the individual `run_tf_1`, `run_tf_2`, and `run_tf_3` scripts have been condensed into a single, dynamic program (`run_tf_N_channel_plume_updraft_day_predict.py`) that automatically identifies the required number of input channels by reviewing the model checkpoint directory.

A significant overhaul was applied to the AACP post-processing methodology. Previously, the model occasionally produced spurious AACP detections along the edges of convective cells or overly contoured the updraft cores themselves. To mitigate this, new physically based constraints were introduced. AACP detections must now fall within a specified distance of a detected OT to be retained. Additional morphological checks ensure the plume maintains a coherent shape, and larger distance padding is used to identify and filter out cloud-edge artifacts. Because of these rigorous clean-up steps, false alarms have been drastically reduced. **Consequently, we highly recommend users rely on the final AACP Object IDs rather than raw AACP probabilities**. This cleaner

post-processing has allowed us to lower the optimal AACP probability threshold (reduced from 0.45 to 0.3) to capture faint plumes, relying on the spatial filters to remove the accompanying noise.

Optimizations to improve run time were made throughout the software. For archived software runs, new parallel processing scripts were introduced to cache longitudes, latitudes, and indices. This prevents the software from re-reading intermediate files on every single iteration, improving processing speed by approximately 3x. Post-processing is now also executed in parallel when in archive mode, and the combined netCDF creation utilizes increased parallel pools for faster writing.

Several bug fixes and user utilities were added. A bug that inadvertently deleted labeled files across all directories has been fixed, which now allows Users to safely run the software for multiple different dates simultaneously on the same machine. A minor synchronization issue was resolved for single-file archive runs to ensure background netCDF writing completes before post-processing begins. To aid in setup, a new utility script (`extract_bbox_coordinates.py`) was created that allows users to quickly identify longitude and latitude boundaries for sub-domain processing by simply drawing a bounding box on a map.

10.1.10 *8 September 2026 VERSION 5.0.0 Release*

VERSION 5.0.0 of the software!!! This major release introduces comprehensive support for EUMETSAT SEVIRI data to detect Overshooting Tops (OTs). The underlying processing architecture was massively overhauled to leverage the `satpy` Python module. Previously, the software relied on single-file netCDF conversions and static Cartesian projection files to map European and African satellite data. By utilizing `satpy`, the software now reads raw, native binary (`.nat`) EUMETSAT files directly. `satpy` natively handles dynamic geographic coordinate mapping and in-memory subsetting (cropping) without relying on heavy external lookup tables. While initially only applied to SEVIRI in order to provide software to partner, this will also be applied for future updates with MTG and GOES. This will result in much faster processing times.

A new "Smart Router" was implemented for SEVIRI EUMETSAT Data Store downloads. Because MSG satellites frequently change missions (e.g., shifting from the 0-degree Prime service over Europe to the 41.5-degree Indian Ocean Data Coverage [IODC] service), users no longer

need to know which satellite was stationed where. By simply entering the satellite name (e.g., `msg1`, `msg2`, `seviri`, `seviri-iodc`) and a target date, the downloader automatically calculates the historical mission timeline and routes the API request to the exact correct EUMETSAT data collection.

To support the satpy library, the software's core dependencies were completely modernized. The environment was upgraded to Python 3.10 and TensorFlow 2.15. Because of these structural changes, a completely new conda environment (`svrstormsig_v5`) is required.

Several critical performance enhancements and bug fixes were implemented for the processing pipeline. A massive computational bottleneck as the software now bypasses expensive, per-pixel Cartopy curved-Earth mathematical transformations and instead utilizes a fast cached projection array.

10.2 *MINOR REVISIONS*

10.2.1 *13 June 2023*

Fixed issue where user wants to correct the model inputs or model type and not everything like the correct optimal model to be called followed with it.

Pass optimal_threshold for a given model to be written to output netCDF file rather than whatever is chosen by user for post-processing and plotting. The one chosen by the user is still passed into the subroutines for plotting and post-processing and is referred to as the likelihood_threshold in order to differentiate it from the optimal_thresh attribute in the raw model likelihood output variables. Users can identify objects in post-processing by specifying their own likelihood thresholds, rather than being forced to use the optimal thresholds found for a particular model.

Decrease size of files by improving compression by setting least_significant_digits keyword when creating the variables.

10.2.2 *21 August 2023*

In post-processing, some users experienced a 'Runtime Warning:invalid value encountered in cast' when initializing the ot_id array. This was likely due to differences in OS's. Numpy is aware of the issue but the line of code has been changed to ot_id = np.full_like(res, 0).astype('uint16'). This

should remove the issue for those users.

ReadME documentation was updated for Google Cloud and includes additional ways to download GOES L1b data for ingestion into the software.

10.2.3 *2 May 2024*

Added proj4 string as a global variable to the output netCDF files to assist with users with their own visualization. Earlier releases contained this information in the imager_projection variable but this string was requested to make plotting easier.

10.2.4 *20 May 2025*

VERSION 3.1 of software. Added post-processing automated quality control for AACP (primarily) and OTs. AACPs < 0.7 removed (below AACP threshold) if not near a cold spot. AACPs <0.7 are also removed if along the border of a convective cell. This removes some good detections but removes many FARs. For OTs, we remove detections where the mean anvil brightness temperature difference ≥ 0 (OT region warmer than surrounding anvil). Lastly, we removed VIS+TROPDIFF OTs within a 64-8:64+8x64-8:64+8 pixel upper and right hand image border surrounding the 2000x2000 mesoscale images. It was found that there were spurious detections along this border. One last change. It seems that the rda.ucar no longer is being updated with tropopause files. The new methods check this and downloads data from Google Cloud instead.

10.2.5 *28 April 2026*

VERSION 4.0.1 of software. Reduced default number of threads for multiprocessing from 3 to 2. Users can go in and change the numbers themselves but to avoid issues we are setting it to 2 for now. See Section 13.6 for more information. Also, adding this updated README file.

10.2.6 *07 May 2026*

VERSION 4.0.2 of software. Due to ability to run post-processing of OTs and AACPs simultaneously, setup the ability to do this for real-time runs. Real-time runs previously post-processed OTs and then AACPs but now it waits until after AACP model is complete and the post-processes both

variables simultaneously when the user specifies to run the software to detect both OT and AACP signatures. Added OT post-processing to use GFS CAPE. Checks that GFS CAPE is $\geq$ 100 J/KG within $\sim$83 km radius in order to retain OT detection. This threshold removes potentially faulty detections in high altitude, thick bumpy cirrus.

10.2.7 *12 June 2026*

VERSION 4.0.3 of software. Fixed issue that occurred when processing a directory containing satellite scans that span multiple UTC days (crossing midnight) while GLM processing is disabled (no_write_glm=True), the software crashed with a KeyError: Index(['glm_files']). This occurred because the mid-loop CSV writer conditionally skipped merging df_glm2, but the subsequent drop_duplicates operation explicitly required the glm_files column to exist.

11 *Authors*

This software was produced through collaborative work between John W. Cooney, Kristopher M. Bedka, and Charles A. Liles at NASA Langley Research Center.

11.1 *Contact Information*

This software is intended for research and operational use. Users can contact the software team regarding its use, especially before publication or public presentation. This is the first official release of this software; these products that are still undergoing validation, testing, quality control, and debugging. Users are invited to address questions and provide feedback to the contacts below.

1. **John Cooney:** john.w.cooney@nasa.gov
2. **Kristopher Bedka:** kristopher.m.bedka@nasa.gov

12 *Disclaimer and Copyright Notices*

12.1 *Notices*

Copyright 2023 United States Government as represented by the Administrator of the National Aeronautics and Space Administration. All Rights Reserved.

12.2 *Disclaimers*

No Warranty: THE SUBJECT SOFTWARE IS PROVIDED ”AS IS” WITHOUT ANY WARRANTY OF ANY KIND, EITHER EXPRESSED, IMPLIED, OR STATUTORY, INCLUDING, BUT NOT LIMITED TO, ANY WARRANTY THAT THE SUBJECT SOFTWARE WILL CONFORM TO SPECIFICATIONS, ANY IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, OR FREEDOM FROM INFRINGEMENT, ANY WARRANTY THAT THE SUBJECT SOFTWARE WILL BE ERROR FREE, OR ANY WARRANTY THAT DOCUMENTATION, IF PROVIDED, WILL CONFORM TO THE SUBJECT SOFTWARE. THIS AGREEMENT DOES NOT, IN ANY MANNER, CONSTITUTE AN ENDORSEMENT BY GOVERNMENT AGENCY OR ANY PRIOR RECIPIENT OF ANY RESULTS, RESULTING DESIGNS, HARDWARE, SOFTWARE PRODUCTS OR ANY OTHER APPLICATIONS RESULTING FROM USE OF THE SUBJECT SOFTWARE. FURTHER, GOVERNMENT AGENCY DISCLAIMS ALL WARRANTIES AND LIABILITIES REGARDING THIRD-PARTY SOFTWARE, IF PRESENT IN THE ORIGINAL SOFTWARE, AND DISTRIBUTES IT ”AS IS.”

12.3 *Waiver and Indemnity*

RECIPIENT AGREES TO WAIVE ANY AND ALL CLAIMS AGAINST THE UNITED STATES GOVERNMENT, ITS CONTRACTORS AND SUBCONTRACTORS, AS WELL AS ANY PRIOR RECIPIENT. IF RECIPIENT’S USE OF THE SUBJECT SOFTWARE RESULTS IN ANY LIABILITIES, DEMANDS, DAMAGES, EXPENSES OR LOSSES ARISING FROM SUCH USE, INCLUDING ANY DAMAGES FROM PRODUCTS BASED ON, OR RESULTING FROM, RECIPIENT’S USE OF THE SUBJECT SOFTWARE, RECIPIENT SHALL INDEMNIFY AND HOLD HARMLESS THE UNITED STATES GOVERNMENT, ITS CONTRACTORS AND SUBCONTRACTORS, AS WELL AS ANY PRIOR RECIPIENT, TO THE EXTENT PERMITTED BY LAW. RECIPIENT’S SOLE REMEDY FOR ANY SUCH MATTER SHALL BE THE IMMEDIATE, UNILATERAL TERMINATION OF THIS AGREEMENT.

13 *Appendix*

13.1 *Allowed Satellites*

This software is capable of being used for the following satellites: MTG-1, GOES-16, GOES-17, GOES-18, and GOES-19.

13.2 *Anaconda Installation*

This following anaconda installation steps are primarily useful for Mac users. Please see Windows Anaconda Install for better steps on how to install anaconda using Windows. Note that the steps refer to the anaconda packages, NOT Anaconda-Navigator.app.

1. Go to: Anaconda Install Website
2. Click download. (This will take you to bottom of web page)
3. Install the 64-Bit Graphical Installer for your system
4. Locate your download and double click it
5. NOTE: when you install Anaconda, it modifies your .bash_profile (this can be important for later)
6. Agree to License Agreement and install anaconda
7. Once complete, close the installer and move it to the trash
8. Activate conda by going to the command line within terminal source <path toconda>/bin/activate. The <path toconda> can change. The default location for Anaconda is:

```
Linux = /home/<your_username>/Anaconda3
Windows = C:\Users\<your_username>\Anaconda3
Mac = /Users/<your_username>/Anaconda3
```

For Mac, if anaconda was installed at the system level, the anaconda package may be located in your /opt directory. If you cannot find the package and, specifically, the anaconda3 directory + bin subdirectory folder then go into your finder and search for anaconda. This should give you the location.

9. In terminal enter *conda init* (this may or may not yield a successful output message).

13.3 *MTG and SEVIRI Satellite Setup*

1. Go to: EUMETSAT API KEY

2. If you have an account, login. If you do not have an account, you will need to register one.

3. Once you have an account, copy the contents from 'Consumer key' and 'Consumer secret' to the bottom of your .bashrc, .zshrc, or whatever it is that you use so that the document looks similar to the following where the contents that you copied are respectively contained within the quotation marks.

   export CONSUMER_KEY="your_consumer_key_goes_here"

   export CONSUMER_SECRET="your_consumer_secret_goes_here"

4. Close Terminal window and open a new one. This should now allow you to use the python functions run_download_mtg_fci_data_parallel.py and run_download_mtg_fci_data.py.

5. If you run into issues using .bashrc, .zshrc etc. an alternative method would be to copy the contents from 'Consumer key' and 'Consumer secret' from EUMETSAT API KEY into lines 93-94 and 67-68 in the python functions run_download_mtg_fci_data_parallel.py and run_download_mtg_fci_data.py, respectively. An example is shown below. In each of those 2 functions (run_download_mtg_fci_data_parallel.py and run_download_mtg_fci_data.py), replace:

   consumer_key = os.getenv("CONSUMER_KEY")

   consumer_secret = os.getenv("CONSUMER_SECRET")

   with the following:

consumer_key = “your_consumer_key_goes_here”

consumer_secret = “your_consumer_secret_goes_here”

13.4 *netCDF File Contents*

13.4.1 *GLM Gridder netCDF File*

Example GLM gridder file is shown below.

```
netcdf 20191201800333_gridded_data {
dimensions:
 x = 2500 ;
 y = 1500 ;
 dim_0 = 1 ;
variables:
 short x(x) ;
  x:_FillValue = -999s ;
  x:axis = "X" ;
  x:long_name = "GOES fixed grid projection x-coordinate" ;
  x:standard_name = "projection_x_coordinate" ;
  x:units = "rad" ;
  x:add_offset = -0.151844 ;
  x:scale_factor = 5.6e-05 ;
 short y(y) ;
  y:_FillValue = -999s ;
  y:axis = "Y" ;
  y:long_name = "GOES fixed grid projection y-coordinate" ;
  y:standard_name = "projection_y_coordinate" ;
  y:units = "rad" ;
  y:add_offset = 0.151844 ;
  y:scale_factor = -5.6e-05 ;
 int goes_imager_projection ;
  goes_imager_projection:long_name = "GOES-R ABI fixed grid projection" ;
  goes_imager_projection:grid_mapping_name = "geostationary" ;
  goes_imager_projection:perspective_point_height = 35786023. ;
  goes_imager_projection:semi_major_axis = 6378137. ;
  goes_imager_projection:semi_minor_axis = 6356752.31414 ;
  goes_imager_projection:inverse_flattening = 298.2572221 ;
  goes_imager_projection:latitude_of_projection_origin = 0. ;
  goes_imager_projection:longitude_of_projection_origin = -75. ;
  goes_imager_projection:sweep_angle_axis = "x" ;
 byte DQF(y, x) ;
  DQF:_FillValue = -1b ;
  DQF:grid_mapping = "goes_imager_projection" ;
  DQF:number_of_qf_values = 6 ;
  DQF:units = "1" ;
  DQF:standard_name = "status_flag" ;
  DQF:long_name = "GLM data quality flags" ;
  DQF:flag_values = 0, 1 ;
  DQF:flag_meanings = "valid, invalid" ;
  DQF:_Unsigned = "true" ;
 double nominal_satellite_subpoint_lat ;
  nominal_satellite_subpoint_lat:_FillValue = -999. ;
  nominal_satellite_subpoint_lat:long_name = "nominal satellite subpoint latitude (platform latitude)" ;
  nominal_satellite_subpoint_lat:standard_name = "latitude" ;
  nominal_satellite_subpoint_lat:units = "degrees_north" ;
 double nominal_satellite_subpoint_lon(dim_0) ;
```

```
 nominal_satellite_subpoint_lon:_FillValue = -999. ;
 nominal_satellite_subpoint_lon:long_name = "nominal satellite subpoint longitude (platform longitude)" ;
 nominal_satellite_subpoint_lon:standard_name = "longitude" ;
 nominal_satellite_subpoint_lon:units = "degrees_east" ;
short flash_extent_density(y, x) ;
 flash_extent_density:_FillValue = 0s ;
 flash_extent_density:standard_name = "flash_extent_density" ;
 flash_extent_density:long_name = "Flash extent density" ;
 flash_extent_density:units = "Count per nominal    3136 microradian^2 pixel per 1.0 min" ;
 flash_extent_density:grid_mapping = "goes_imager_projection" ;
 flash_extent_density:add_offset = 0. ;
 flash_extent_density:scale_factor = 0.0625 ;
 flash_extent_density:_Unsigned = "true" ;
short flash_centroid_density(y, x) ;
 flash_centroid_density:_FillValue = 0s ;
 flash_centroid_density:standard_name = "flash_centroid_density" ;
 flash_centroid_density:long_name = "Flash centroid density" ;
 flash_centroid_density:units = "Count per nominal    3136 microradian^2 pixel per 1.0 min" ;
 flash_centroid_density:grid_mapping = "goes_imager_projection" ;
 flash_centroid_density:add_offset = 0. ;
 flash_centroid_density:scale_factor = 1. ;
 flash_centroid_density:_Unsigned = "true" ;
short average_flash_area(y, x) ;
 average_flash_area:_FillValue = 0s ;
 average_flash_area:standard_name = "average_flash_area" ;
 average_flash_area:long_name = "Average flash area" ;
 average_flash_area:units = "km^2 per flash" ;
 average_flash_area:grid_mapping = "goes_imager_projection" ;
 average_flash_area:add_offset = 0. ;
 average_flash_area:scale_factor = 10. ;
 average_flash_area:_Unsigned = "true" ;
short total_energy(y, x) ;
 total_energy:_FillValue = 0s ;
 total_energy:standard_name = "total_energy" ;
 total_energy:long_name = "Total radiant energy" ;
 total_energy:units = "nJ" ;
 total_energy:grid_mapping = "goes_imager_projection" ;
 total_energy:add_offset = 0. ;
 total_energy:scale_factor = 1.52597e-06 ;
 total_energy:_Unsigned = "true" ;
short group_extent_density(y, x) ;
 group_extent_density:_FillValue = 0s ;
 group_extent_density:standard_name = "group_extent_density" ;
 group_extent_density:long_name = "Group extent density" ;
 group_extent_density:units = "Count per nominal    3136 microradian^2 pixel per 1.0 min" ;
 group_extent_density:grid_mapping = "goes_imager_projection" ;
 group_extent_density:add_offset = 0. ;
 group_extent_density:scale_factor = 0.25 ;
 group_extent_density:_Unsigned = "true" ;
short group_centroid_density(y, x) ;
 group_centroid_density:_FillValue = 0s ;
 group_centroid_density:standard_name = "group_centroid_density" ;
 group_centroid_density:long_name = "Group centroid density" ;
 group_centroid_density:units = "Count per nominal    3136 microradian^2 pixel per 1.0 min" ;
 group_centroid_density:grid_mapping = "goes_imager_projection" ;
 group_centroid_density:add_offset = 0. ;
 group_centroid_density:scale_factor = 1. ;
 group_centroid_density:_Unsigned = "true" ;
short average_group_area(y, x) ;
 average_group_area:_FillValue = 0s ;
 average_group_area:standard_name = "average_group_area" ;
```

```
  average_group_area:long_name = "Average group area" ;
  average_group_area:units = "km^2 per group" ;
  average_group_area:grid_mapping = "goes_imager_projection" ;
  average_group_area:add_offset = 0. ;
  average_group_area:scale_factor = 1. ;
  average_group_area:_Unsigned = "true" ;
 short minimum_flash_area(y, x) ;
  minimum_flash_area:_FillValue = 0s ;
  minimum_flash_area:standard_name = "minimum_flash_area" ;
  minimum_flash_area:long_name = "Minimum flash area" ;
  minimum_flash_area:units = "km^2" ;
  minimum_flash_area:grid_mapping = "goes_imager_projection" ;
  minimum_flash_area:add_offset = 0. ;
  minimum_flash_area:scale_factor = 10. ;
  minimum_flash_area:_Unsigned = "true" ;

// global attributes:
  :cdm_data_type = "Image" ;
  :Conventions = "CF-1.7" ;
  :id = "93cb84a3-31ef-4823-89f5-c09d88fc89e8" ;
  :institution = "DOC/NOAA/NESDIS > U.S. Department of Commerce, National Oceanic and Atmospheric Administration, National Environmental Satellite, Dat
  :instrument_type = "GOES R Series Geostationary Lightning Mapper" ;
  :iso_series_metadata_id = "f5816f53-fd6d-11e3-a3ac-0800200c9a66" ;
  :keywords = "ATMOSPHERE > ATMOSPHERIC ELECTRICITY > LIGHTNING, ATMOSPHERE > ATMOSPHERIC PHENOMENA > LIGHTNING" ;
  :keywords_vocabulary = "NASA Global Change Master Directory (GCMD) Earth Science Keywords, Version 7.0.0.0.0" ;
  :license = "Unclassified data.  Access is restricted to approved users only." ;
  :Metadata_Conventions = "Unidata Dataset Discovery v1.0" ;
  :naming_authority = "gov.nesdis.noaa" ;
  :processing_level = "National Aeronautics and Space Administration (NASA) L2" ;
  :project = "GOES" ;
  :standard_name_vocabulary = "CF Standard Name Table (v25, 05 July 2013)" ;
  :summary = "The Lightning Detection Gridded product generates fields starting from the GLM Lightning Detection Events, Groups, Flashes product.  It c
  :title = "GLM L2 Lightning Detection Gridded Product" ;
  :dataset_name = "OR_GLM-L2-GLMC-M3_G16_s20191201801400_e20191201802400_c20203501353540.nc" ;
  :date_created = "2020-12-15T13:53:54.935958Z" ;
  :instrument_ID = "FM1" ;
  :orbital_slot = "GOES-East" ;
  :platform_ID = "G16" ;
  :production_data_source = "Postprocessed" ;
  :production_environment = "DE" ;
  :production_site = "TTU" ;
  :scene_id = "CONUS" ;
  :spatial_resolution = "2km at nadir" ;
  :time_coverage_end = "2019-04-30T18:02:40Z" ;
  :time_coverage_start = "2019-04-30T18:01:40Z" ;
  :timeline_id = "ABI Mode 3" ;
}
```

Size of file depends on how many GLM files are used to create this gridded file. A single gridded file is created for all of the raw GLM data files in specified directory. Creating the gridded GLM files also takes a good chunk of time to do so it might be beneficial for the user to create these files prior to running main script. As long as the files are named using a similar convention to that in the main program, there should not be any issue reading it in.

13.4.2 *Combine IR, VIS, and GLM Data netCDF File*

Example IR, VIS, and GLM combined netCDF file is shown below.

```
netcdf OR_ABI_L1b_M2_COMBINED_s20231692357560_e20231692358033_c20231692358050 {
dimensions:
 Y = 2000 ;
 X = 2000 ;
 time = 1 ;
variables:
 float longitude(Y, X) ;
  longitude:least_significant_digit = 3LL ;
  longitude:long_name = "longitude -180 to 180 degrees east" ;
  longitude:standard_name = "longitude" ;
  longitude:units = "degrees_east" ;
 float latitude(Y, X) ;
  latitude:least_significant_digit = 3LL ;
  latitude:long_name = "latitude -90 to 90 degrees north" ;
  latitude:standard_name = "latitude" ;
  latitude:units = "degrees_north" ;
 float time(time) ;
  time:long_name = "J2000 epoch mid-point between the start and end image scan in seconds" ;
  time:standard_name = "time" ;
  time:units = "seconds since 2000-01-01 12:00:00" ;
 int visible_reflectance(time, Y, X) ;
  visible_reflectance:_FillValue = -2147483648 ;
  visible_reflectance:long_name = "Visible Reflectance Normalized by Solar Zenith Angle" ;
  visible_reflectance:standard_name = "Visible_Reflectance" ;
  visible_reflectance:units = "reflectance_normalized_by_solar_zenith_angle" ;
  visible_reflectance:coordinates = "longitude latitude time" ;
  visible_reflectance:add_offset = 1.009126f ;
  visible_reflectance:scale_factor = 4.614115e-10f ;
 int solar_zenith_angle(time, Y, X) ;
  solar_zenith_angle:_FillValue = -2147483648 ;
  solar_zenith_angle:least_significant_digit = 2LL ;
  solar_zenith_angle:long_name = "Solar Zenith Angle" ;
  solar_zenith_angle:standard_name = "solar_zenith_angle" ;
  solar_zenith_angle:units = "degrees" ;
  solar_zenith_angle:add_offset = 76.50061f ;
  solar_zenith_angle:scale_factor = 3.591184e-09f ;
 int imager_projection ;
  imager_projection:long_name = "GOES-R ABI fixed grid projection" ;
  imager_projection:satellite_name = "G16" ;
  imager_projection:grid_mapping_name = "geostationary" ;
  imager_projection:perspective_point_height = 35786023. ;
  imager_projection:semi_major_axis = 6378137. ;
  imager_projection:semi_minor_axis = 6356752.31414 ;
  imager_projection:inverse_flattening = 298.2572221 ;
  imager_projection:latitude_of_projection_origin = 0. ;
  imager_projection:longitude_of_projection_origin = -75. ;
  imager_projection:sweep_angle_axis = "x" ;
  imager_projection:bounds = "-1923606.125,-922098.4375,2665593.5,3667101.25" ;
  imager_projection:bounds_units = "m" ;
 int ir_brightness_temperature(time, Y, X) ;
  ir_brightness_temperature:_FillValue = -2147483648 ;
  ir_brightness_temperature:long_name = "Infrared Brightness Temperature Image resampled onto VIS data grid" ;
  ir_brightness_temperature:standard_name = "IR_brightness_temperature" ;
  ir_brightness_temperature:units = "kelvin" ;
  ir_brightness_temperature:coordinates = "longitude latitude time" ;
  ir_brightness_temperature:add_offset = 254.1493f ;
```

```
  ir_brightness_temperature:scale_factor = 2.284545e-08f ;
 int glm_flash_extent_density(time, Y, X) ;
  glm_flash_extent_density:_FillValue = -2147483648 ;
  glm_flash_extent_density:least_significant_digit = 4LL ;
  glm_flash_extent_density:long_name = "Flash extent density within +/- 2.5 min of time variable resampled onto VIS data grid and then smoothed using G
  glm_flash_extent_density:standard_name = "Flash_extent_density" ;
  glm_flash_extent_density:units = "Count per nominal 3136 microradian^2 pixel per 1.0 min" ;
  glm_flash_extent_density:coordinates = "longitude latitude time" ;
  glm_flash_extent_density:add_offset = 4. ;
  glm_flash_extent_density:scale_factor = 1.86264515009832e-09 ;
 float ir_vis_glm_ot(time, Y, X) ;
  ir_vis_glm_ot:standard_name = "IR+VIS+GLM_OT_Model_Results" ;
  ir_vis_glm_ot:least_significant_digit = 3LL ;
  ir_vis_glm_ot:optimal_thresh = 0.4 ;
  ir_vis_glm_ot:missing_value = 0.f ;
  ir_vis_glm_ot:units = "dimensionless" ;
  ir_vis_glm_ot:coordinates = "longitude latitude time" ;
  ir_vis_glm_ot:valid_range = 0.f, 1.f ;
  ir_vis_glm_ot:checkpoint_file = "/Users/jwcooney/python/code/aacp/data/model_checkpoints/ir_vis_glm/updraft_day_model/2022-02-18/multiresunet/chosen_
  ir_vis_glm_ot:model_type = "Multiresunet" ;
  ir_vis_glm_ot:long_name = "IR+VIS+GLM OT Multiresunet Machine Learning Detection Likelihood" ;
 ushort ir_vis_glm_ot_id_number(time, Y, X) ;
  ir_vis_glm_ot_id_number:description = "The object Identification Number field shows all pixels that belong to an individual object region.  The ID nu
  ir_vis_glm_ot_id_number:standard_name = "IR+VIS+GLM OT ID Number" ;
  ir_vis_glm_ot_id_number:missing_value = 0US ;
  ir_vis_glm_ot_id_number:units = "dimensionless" ;
  ir_vis_glm_ot_id_number:likelihood_threshold = 0.4 ;
  ir_vis_glm_ot_id_number:coordinates = "longitude latitude time" ;
  ir_vis_glm_ot_id_number:model_type = "Multiresunet" ;
  ir_vis_glm_ot_id_number:long_name = "IR+VIS+GLM OT Identification Number" ;
 float ir_vis_glm_ot_anvilmean_brightness_temperature_difference(time, Y, X) ;
  ir_vis_glm_ot_anvilmean_brightness_temperature_difference:description = "Minimum brightness temperature within an OT Minus Anvil Brightness Temperatu
  ir_vis_glm_ot_anvilmean_brightness_temperature_difference:standard_name = "IR+VIS+GLM OT - Anvil BTD" ;
  ir_vis_glm_ot_anvilmean_brightness_temperature_difference:least_significant_digit = 2LL ;
  ir_vis_glm_ot_anvilmean_brightness_temperature_difference:missing_value = NaNf ;
  ir_vis_glm_ot_anvilmean_brightness_temperature_difference:units = "K" ;
  ir_vis_glm_ot_anvilmean_brightness_temperature_difference:likelihood_threshold = 0.4 ;
  ir_vis_glm_ot_anvilmean_brightness_temperature_difference:coordinates = "longitude latitude time" ;
  ir_vis_glm_ot_anvilmean_brightness_temperature_difference:valid_range = -50.f, 0.f ;
  ir_vis_glm_ot_anvilmean_brightness_temperature_difference:model_type = "Multiresunet" ;
  ir_vis_glm_ot_anvilmean_brightness_temperature_difference:long_name = "IR+VIS+GLM Overshooting Top Minus Anvil Brightness Temperature Difference" ;
 float tropopause_temperature(time, Y, X) ;
  tropopause_temperature:standard_name = "GFS Tropopause" ;
  tropopause_temperature:least_significant_digit = 2LL ;
  tropopause_temperature:missing_value = NaNf ;
  tropopause_temperature:units = "K" ;
  tropopause_temperature:coordinates = "longitude latitude time" ;
  tropopause_temperature:valid_range = 160.f, 310.f ;
  tropopause_temperature:long_name = "Temperature of the Tropopause Retrieved from GFS Interpolated and Smoothed onto Satellite Grid" ;

// global attributes:
  :Conventions = "CF-1.8" ;
  string :description = "This file combines unscaled IR data, Visible data, and GLM data into one NetCDF file. The VIS data is on its original grid but
  :geospatial_lat_min = "25.43004" ;
  :geospatial_lat_max = "37.462128" ;
  :geospatial_lon_min = "-98.60013" ;
  :geospatial_lon_max = "-84.41396" ;
  :x_inds = "" ;
  :y_inds = "" ;
  :spatial_resolution = "0.5km at nadir" ;
}
```

These files are each up to $\sim$ 45 MB. A single file is created for each IR/VIS file. The file above scales the data to save space as well as resamples the GLM and IR data to the VIS data grid. The GLM data are smoothed using a Gaussian smoother after resampling the GLM data but prior to writing the netCDF file.

13.5 *GOES Storage Bucket Download Examples*

A few examples of how the GOES-16 IR, VIS, and GLM data can be downloaded. For these examples, the data are downloaded to another Google Cloud Storage Bucket named, 'goes-data'. The data can also be downloaded to local directories as well.

```
gsutil -m cp -r gs://gcp-public-data-goes-16/GLM-L2-LCFA/2019/125/1[8-9]/**** gs://goes-data/20190505-06/glm

gsutil -m cp -r gs://gcp-public-data-goes-16/ABI-L1b-RadM/2019/128/0[0-3]/*C13_G16_* gs://goes-data/20190507-08/ir

gsutil -m cp -r gs://gcp-public-data-goes-16/ABI-L1b-RadM/2019/128/0[0-3]/*C02_G16_* gs://goes-data/20190507-08/vis
```

13.6 *Potential Speed Ups*

13.7 *ProcessPool Adjustments for Creating Combined netCDF Files*

During the generation of combined netCDF files, the software utilizes Python's multiprocessing library (`multiprocessing.Pool`) to process multiple files simultaneously. This significantly reduces the overall time required to read the input modalities and write the combined outputs.

By default, the software is configured to use 2 parallel worker processes. This default was chosen to ensure stability across a wide range of user hardware and to prevent the system from hanging or freezing on machines with limited processing power or memory constraints.

Potential Speed-Ups for Powerful Machines:

Users running the software on high-performance machines (with numerous CPU cores and ample RAM) can decrease processing time by increasing the number of pools. To do this, modify the `run_create_image_from_three_modalities2.py` and `run_mtg_create_image_from_three_moc` scripts located in the `glm_gridder` subdirectory. Locate the `mp.Pool` initialization lines which look like this:

```
pool = mp.Pool(2, maxtasksperchild = 30)
```

By changing the first argument from `2` to `3` (or higher, depending on your system's specifications), the software will process more files concurrently, yielding a noticeable speed-up. The second argument, `maxtasksperchild`, can also be increased. Increasing that value means the cache information is kept for more files.

Troubleshooting System Hangs:

Conversely, if a User experiences the software hanging indefinitely during the combined netCDF creation step, it is highly likely that the system's compute resources are being overwhelmed by the multiprocessing execution. In this scenario, the User should verify that the pool size in the aforementioned scripts is set to the default of `2`. If the issue persists on particularly slow machines, reducing the pool size to `1` will alleviate hardware-related bottlenecks at the cost of slower execution times.

13.8 *ProcessPoolExecutor Adjustments for Post-Processing*

When running the software in archive mode over a date range that spans multiple days, the post-processing step (which calculates BTD, filters AACPs, and interpolates GFS tropopause temperatures) is executed in parallel. The software assigns each day to a separate worker to process the netCDF files simultaneously.

By default, the software restricts this parallel execution to 2 concurrent workers using Python's `concurrent.futures.ProcessPoolExecutor`. This conservative default prevents the system from running out of memory (RAM) when holding multiple large numpy arrays and satellite grids at once.

Potential Speed-Ups for Powerful Machines:

If you are running the software on a robust machine or a high-performance computing cluster, you can significantly decrease the time it takes to post-process multi-day archived data by increasing the number of workers.

To do this, locate the main prediction script (`run_all_plume_updraft_model_predict.py` in the `new_model` subdirectory) and find the `ProcessPoolExecutor` block near the end of the script:

```
if len(postproc_tasks) > 0:
    print(f"Launching post-processing for {len(postproc_tasks)} day(s).
```

```
with concurrent.futures.ProcessPoolExecutor(max_workers=2) as execut
    results = executor.map(_parallel_post_process, postproc_tasks)
```

Change the `max_workers=2` argument to a higher number (e.g., `3`, `4`, or `5`) depending on your system's available CPU cores and memory. The script will then process that many days simultaneously.

Troubleshooting System Hangs or Memory Errors:

Because post-processing involves memory-intensive operations like morphological dilations and distance transforms, setting `max_workers` too high may result in Out-Of-Memory (OOM) errors or cause the system to freeze. If the script hangs during the "Launching post-processing..." phase, or if the terminal is killed by the operating system, revert `max_workers` back to `2`. If the machine is exceptionally constrained, setting `max_workers=1` will force the software to process days sequentially, avoiding memory overload.

14 References Cited

Bruning, E. C., et al., 2019: Meteorological imagery for the geostationary lightning mapper. *Journal of Geophysical Research: Atmospheres*, **124 (24)**, 14 285–14 309.

Cooney, J. W., K. M. Bedka, C. A. Liles, and C. R. Homeyer, 2025: Automated detection of overshooting tops and above-anvil cirrus plumes within geostationary imagery using deep learning. *Artif. Intell. Earth Syst.*, **4**.

Google, ????: gcp-public-data-goes-16. https://console.cloud.google.com/marketplace/product/noaa-public/goes.

Schmit, T. J., P. Griffith, M. M. Gunshor, J. M. Daniels, S. J. Goodman, and W. J. Lebair, 2017: A closer look at the abi on the goes-r series. *Bulletin of the American Meteorological Society*, **98**, 681–698.

Schmit, T. J., S. S. Lindstrom, J. J. Gerth, and M. M. Gunshor, 2018: Applications of the 16 spectral bands on the advanced baseline imager (abi). *J. Operational Meteor.*, **6 (4)**, 33–46.

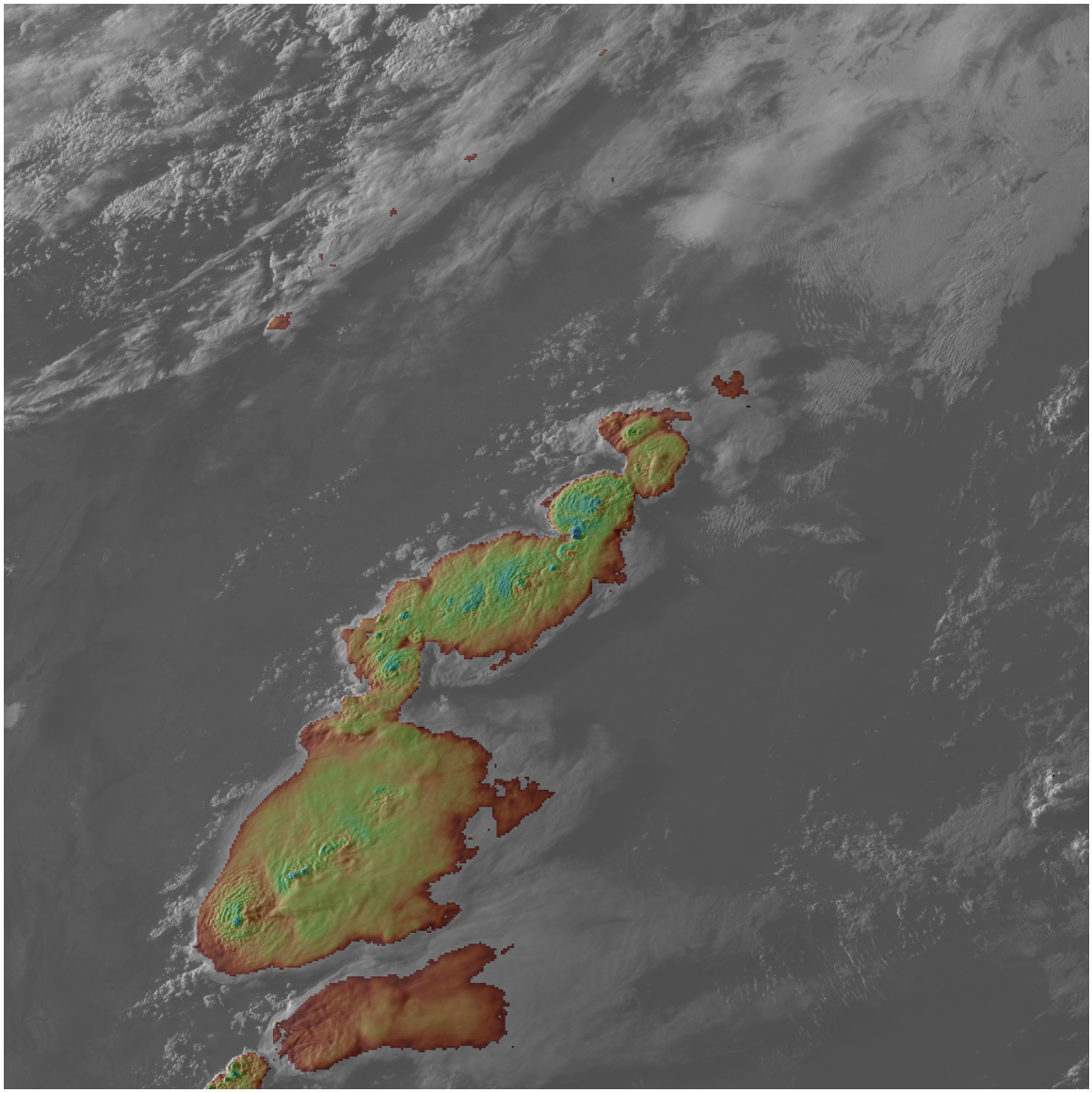


Figure 1: Example IR/VIS image for combined netCDF file: OR_ABI_L1b_M2_COMBINED_s20201350016495_e20201350016564_c20201350017016.

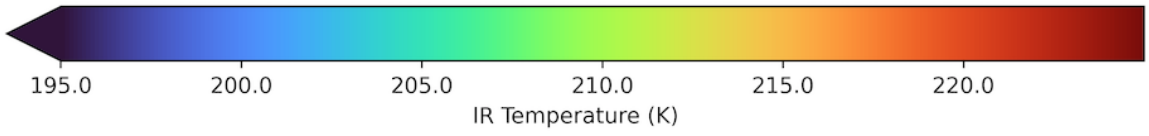


Figure 2: IR color bar.

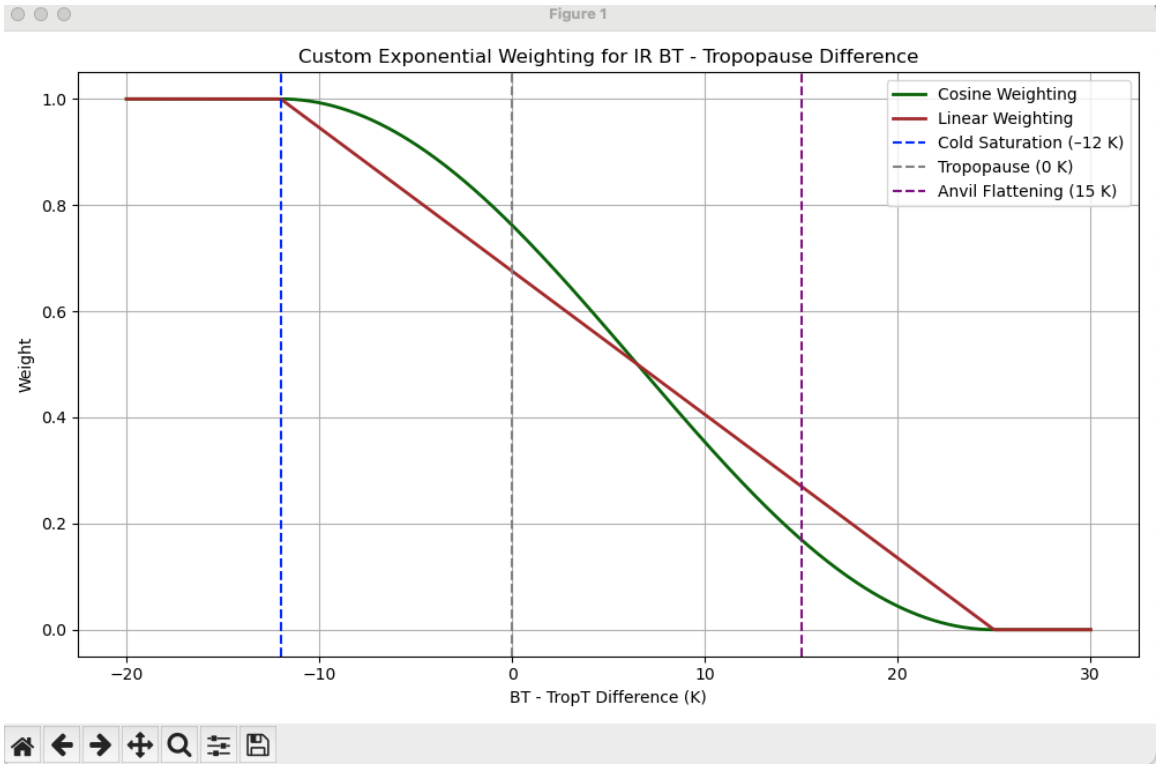


Figure 3: Difference in weighting at each level between using a linear weighting function for TROPDIFF and a cosine weighting function.